

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO MÔN DỰ ÁN KHOA HỌC
NGÀNH: KHOA HỌC MÁY TÍNH

ĐỀ TÀI:

SINH VIÊN: NGUYỄN MẠNH QUỲNH

MSV: 22028241

Lớp: INT3132 2

Giảng viên hướng dẫn: PGS.TS. Võ Đình Hiếu

Hà Nội, tháng 12 năm 2025

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn chân thành và sâu sắc đến Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội cùng toàn thể các thầy cô trong khoa. Trong suốt quá trình học tập, các thầy cô đã không chỉ truyền đạt cho em những kiến thức chuyên môn quý báu mà còn tạo điều kiện thuận lợi để em được tham gia kỳ thực tập này. Đây là nền tảng quan trọng giúp em tự tin tiếp cận môi trường làm việc thực tế.

Đặc biệt, em xin gửi lời cảm ơn sâu sắc đến tới PGS.TS. Võ Đình hiếu– giảng viên hướng dẫn em trong học phần Dự án công nghệ. Trong suốt thời gian tham gia học phần này, em đã nhận được sự giảng dạy tận tình và những định hướng quý báu từ thầy. Những kiến thức nền tảng mà thầy truyền đạt không chỉ giúp em hoàn thành báo cáo cuối kỳ này mà còn là hành trang quan trọng cho sau này.

Do kiến thức và kinh nghiệm thực tế của bản thân còn hạn chế, báo cáo chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ thầy để em có thể hoàn thiện hơn nữa kiến thức cũng như kỹ năng trong tương lai.

Em xin chân thành cảm ơn!

Mục lục

Contents

Chương 1 - Giới thiệu	5
1.1. Đặt vấn đề	5
1.2. Mục tiêu dự án	5
1.2.1. Mục tiêu chức năng.....	5
1.3. Phạm vi nghiên cứu	6
1.3.1. Công nghệ sử dụng	6
1.3.2. Về chức năng	7
1.4. Cấu trúc báo cáo.....	7
Chương 2 - Cơ sở lý thuyết.....	9
2.1. Sơ đồ tư duy.....	9
2.1.1. Khái niệm.....	9
2.1.2. Các thành phần cơ bản	9
2.2. Backend và RESTful API	9
2.3. Frontend với React và ReactFlow.....	10
2.3.1. React	10
2.3.2. React Flow – Xử lý đồ thị Mindmap	10
2.3.3. Luồng xử lý dữ liệu Frontend	10
2.4. Real-time Collaboration với WebSocket và Yjs.....	11
2.5. OpenAI GPT Model.....	12
2.6. Cơ sở dữ liệu.....	13
Chương 3 - Phân tích và thiết kế hệ thống.....	14
3.1. Phân tích yêu cầu hệ thống	14
3.1.1. Yêu cầu chức năng (Functional Requirements)	14
3.1.2. Yêu cầu phi chức năng (Non-Functional Requirements).....	16
3.2. Kiến trúc hệ thống.....	18
3.2.1. Kiến trúc tổng thể.....	18
3.2.2. Luồng xử lý chính	18
3.2.3. Giao tiếp giữa các thành phần.....	20
3.3. Thiết kế Use Case	21
3.3.1. Danh sách các Use Case chính.....	22

3.3.2. Use Case Chi tiết.....	22
3.4. Biểu đồ tuần tự (Sequence Diagrams)	46
3.5. Thiết kế cơ sở dữ liệu.....	60
3.5.1. Tổng quan Database.....	60
3.5.2. User Collection	60
3.5.3. Mindmap Collection	60
3.5.4. Version Collection	61
3.5.5. Notification Collection.....	62
3.5.6. RefreshToken Collection	63
3.5.7. AuditLog Collection	64
3.5.8. Database Relationships Diagram	64
3.6. Thiết kế giao diện UI	65
3.6.1. Đăng nhập, đăng ký	65
3.6.2. Dashboard	65
3.6.3. Profile Page.....	65
3.6.4. Create Mindmap popup và Mindmap Card ở Dashboard	66
3.6.5. Editor Page khi bấm vào MindmapCard.....	66
3.6.6. Save version popup, thông báo và history	67
3.6.7. Popup AI Assistant	67
3.6.8. Thanh công cụ của Node.....	67
3.6.9. Chỉnh custom connectioon.....	68
Chương 4 : Triển khai hệ thống	69
4.1. Môi trường phát triển.....	69
4.1.1. Hệ điều hành	69
4.1.2. Công nghệ và thư viện chính	69
4.1.3. Công cụ hỗ trợ phát triển	70
4.2. Giao diện người dùng (Đã có ở phần thiết kế).....	70
Chương 5: Kết luận và hướng phát triển.....	71
5.1. Kết luận.....	71
5.2. Định hướng phát triển	71
Tài liệu tham khảo	73

Chương 1 - Giới thiệu

1.1. Đặt vấn đề

Trong thời đại công nghệ số phát triển mạnh mẽ, việc tổ chức và trực quan hóa thông tin đóng vai trò quan trọng trong học tập và làm việc. Sơ đồ tư duy (mind mapping) là cách thức giúp người dùng sắp xếp ý tưởng hiệu quả, lập kế hoạch dự án, ghi chú học tập. Tuy nhiên, các giải pháp hiện có trên thị trường còn nhiều hạn chế:

- **Thiếu tính cộng tác thời gian thực:** Các công cụ truyền thống như XMind chưa, hoặc hạn chế làm việc nhóm đồng thời.
- **Giao diện phức tạp:** Nhiều ứng dụng có quá nhiều tính năng làm người dùng mới gặp khó khăn khi tiếp cận như Miro.
- **Thiếu hỗ trợ AI:** Việc tạo sơ đồ tư duy từ đầu mất nhiều thời gian, đặc biệt với những người chưa quen với phương pháp này.
- **Quản lý phiên bản kém:** Khó khăn trong việc theo dõi lịch sử thay đổi và khôi phục về phiên bản trước.

Nhận thấy những vấn đề trên, dự án này được thực hiện nhằm xây dựng trang web cho phép vẽ sơ đồ tư duy hiện đại, tích hợp công nghệ real-time collaboration và trí tuệ nhân tạo, mang đến trải nghiệm người dùng tốt hơn với chi phí hợp lý.

1.2. Mục tiêu dự án

1.2.1. Mục tiêu chức năng

1.2.1.1. Quản lý người dùng và xác thực

- Đăng ký, đăng nhập với JWT authentication
- Quản lý profile, đổi mật khẩu, upload avatar
- Refresh token tự động để duy trì session

1.2.1.2. Tạo và quản lý sơ đồ tư duy

- Tạo mindmap mới với tiêu đề và mô tả
- Thêm, sửa, xóa nodes.
- Tạo connections (liên kết) giữa các nodes
- Tùy chỉnh màu sắc, font chữ, style cho từng node
- Hỗ trợ cả parent-child relationship và custom connections

1.2.1.3. Cộng tác thời gian thực

- Nhiều người dùng chỉnh sửa cùng lúc trên cùng một mindmap
- Hiện thị các thành viên đang online
- Đồng bộ thay đổi tức thì không cần reload trang
- Xử lý xung đột tự động với CRDT (Yjs)

1.2.1.4. Quản lý quyền truy cập

- Owner: Toàn quyền quản lý mindmap và team
- Editor: Cho phép chỉnh sửa nội dung và lưu version
- Viewer: Chỉ xem, không sửa
- Gửi lời mời cộng tác

1.2.1.5. Version Control

- Auto-save mỗi 10 giây khi có thay đổi
- Lưu version thủ công với label tùy chỉnh
- Xem lịch sử các versions
- Khôi phục về version cũ (restore)

1.2.1.6. Tích hợp AI cơ bản

- Tự động sinh sơ đồ từ mô tả văn bản (text prompt)
- Đề xuất nodes con dựa trên context hiện tại

1.2.1.7. Hệ thống thông báo

- Thông báo lời mời cộng tác
- Accept/Reject invitation
- Đếm số thông báo chưa đọc

1.3. Phạm vi nghiên cứu

1.3.1. Công nghệ sử dụng

Frontend:

- React - UI framework
- ReactFlow - Thư viện vẽ flowchart/mindmap
- Zustand - State management
- React Query - Data fetching và caching
- Axios - HTTP client
- Tailwind CSS - Utility-first CSS framework

Backend:

- Node.js với Express
- MongoDB với Mongoose ODM
- JWT (jsonwebtoken) cho authentication
- Bcrypt cho mã hóa password
- Multer cho upload file

Realtime Server:

- WebSocket
- Yjs - CRDT framework
- y-websocket - WebSocket provider cho Yjs

AI Service:

- OpenAI API (GPT-4o-mini)

1.3.2. Về chức năng

Đã triển khai:

- Đăng ký, đăng nhập với JWT
- Quản lý profile (update name, avatar, password)
- CRUD mindmaps
- Vẽ sơ đồ với nodes và edges
- Real-time collaboration với awareness (hiển thị cursor)
- Tùy chỉnh style nodes (màu sắc, font, bold/italic/underline)
- Parent-child relationships tự động
- Custom connections với style configurable
- Auto-layout algorithm (cân bằng cây 2 bên)
- Drag & drop nodes, reparenting
- Undo/Redo với Yjs UndoManager
- Version control (auto-save + manual save)
- Restore to previous version
- Collaboration management (invite, remove, change role)
- Notification system (invite accept/reject)
- AI mindmap generation từ text prompt
- AI suggestions cho nodes
- Dashboard với owned/shared mindmaps
- Read-only mode cho viewers

Chưa triển khai (ngoài phạm vi):

- Đăng nhập bằng Google OAuth
- Export sang PDF/PNG
- Hiện thị chuột hoặc node select của người khác
- Import từ các format khác
- Chế độ offline
- Mobile app native (chỉ responsive web)
- Video call tích hợp
- Chat trong app
- Template library

1.4. Cấu trúc báo cáo

Báo cáo được tổ chức thành 6 chương:

Chương 1 - Giới thiệu: Trình bày bối cảnh, mục tiêu và phạm vi của dự án.

Chương 2 - Cơ sở lý thuyết: Giải thích các khái niệm và công nghệ nền tảng được sử dụng trong dự án, bao gồm mind mapping, kiến trúc web hiện đại, real-time collaboration, AI/LLM, và các framework cụ thể.

Chương 3 - Phân tích và thiết kế: Phân tích yêu cầu chi tiết, thiết kế kiến trúc tổng thể, use case diagram, thiết kế database schema, thiết kế API endpoints, và wireframe giao diện.

Chương 4 - Triển khai hệ thống: Mô tả chi tiết quá trình implementation từng module: Frontend (React), Backend (Express), Realtime (WebSocket/Yjs), AI Service (Flask). Trình bày các kỹ thuật quan trọng và giải pháp cho các vấn đề gặp phải.

Chương 5 - Kết quả và đánh giá: Trình bày kết quả đạt được, demo các chức năng chính, đánh giá hiệu năng, so sánh với các sản phẩm tương tự trên thị trường.

Chương 6 - Kết luận: Tổng kết những gì đã học được, đánh giá mức độ hoàn thành mục tiêu, những hạn chế còn tồn tại, và đề xuất hướng phát triển tiếp theo.

Chương 2 - Cơ sở lý thuyết

2.1. Sơ đồ tư duy

2.1.1. Khái niệm

Sơ đồ tư duy là phương pháp trực quan hóa, tổ chức thông tin dưới dạng cấu trúc cây, với một ý tưởng trung tâm (root node) được đặt ở giữa và các nhánh (node) con tỏa ra xung quanh. Phương pháp này giúp người dùng:

- Hệ thống hóa kiến thức một cách trực quan
- Kích thích tư duy liên tưởng và sáng tạo
- Ghi nhớ thông tin hiệu quả hơn so với văn bản tuyến tính

Trong môi trường số, Mindmap thường được biểu diễn bằng các node và edge, cho phép người dùng thao tác trực tiếp như thêm, sửa, xóa, di chuyển node.

2.1.2. Các thành phần cơ bản

- **Center Node hay Root Node (Nút trung tâm):** Chủ đề chính, là gốc của cây tư duy
- **Branches (Nhánh):** Các ý tưởng con xuất phát từ trung tâm
- **Sub-branches (Nhánh phụ):** Chi tiết hóa các branches chính
- **Connections:** Liên kết giữa các ý tưởng không có quan hệ cha-con
- **Colors & Icons:** Phân biệt và làm nổi bật các nhóm ý tưởng

2.2. Backend và RESTful API

REST (Representational State Transfer) là nguyên tắc thiết kế cho web services, sử dụng HTTP methods chuẩn:

- **GET:** Lấy dữ liệu (e.g., GET /api/mindmaps)
- **POST:** Tạo mới (e.g., POST /api/mindmaps)
- **PUT/PATCH:** Cập nhật (e.g., PUT /api/mindmaps/:id)
- **DELETE:** Xóa (e.g., DELETE /api/mindmaps/:id)

Đặc điểm của RESTful API:

- **Stateless:** Server không lưu session, mỗi request độc lập
- **Resource-based:** URL đại diện cho resource (mindmaps, users)
- **JSON format:** Dữ liệu trao đổi dạng JSON
- **HTTP status codes:** 200 (OK), 201 (Created), 404 (Not Found), 500 (Error)

Backend cung cấp các API để:

- Xử lý các yêu cầu từ Frontend gọi đến
- Gọi đến dịch vụ AI
- Tương tác để xử lý các tác vụ liên quan với Realtime service.

2.3. Frontend với React và ReactFlow

2.3.1. React

React được sử dụng để tổ chức ứng dụng theo mô hình component-based, trong đó mỗi thành phần đảm nhiệm một chức năng cụ thể. Các ưu điểm chính của React trong dự án bao gồm:

- Quản lý trạng thái ứng dụng một cách hiệu quả
- Tách biệt logic xử lý và hiển thị
- Dễ dàng tích hợp với thư viện đồng bộ real-time (Yjs)
- Dễ mở rộng và bảo trì mã nguồn

State của frontend được chia thành:

- Local UI state (selection, mode, viewport)
- Shared state (node, edge) được đồng bộ qua Yjs

2.3.2. React Flow – Xử lý đồ thị Mindmap

React Flow đóng vai trò là engine đồ thị cho ứng dụng mindmap. Thư viện này chịu trách nhiệm:

- Render node và edge dựa trên dữ liệu cấu trúc
- Cung cấp cơ chế drag, zoom, pan
- Phát sinh các sự kiện thay đổi đồ thị (add, move, connect, delete)

React Flow hoạt động độc lập với nguồn dữ liệu, cho phép dữ liệu được quản lý bởi Yjs và chỉ sử dụng React Flow như lớp hiển thị và tương tác.

2.3.3. Luồng xử lý dữ liệu Frontend

Luồng hoạt động của frontend được thiết kế như sau:

1. React Flow render sơ đồ từ state hiện tại
2. Người dùng thực hiện thao tác (di chuyển node, tạo edge, chỉnh sửa nội dung)
3. React bắt sự kiện và cập nhật dữ liệu tương ứng
4. Thay đổi được ghi vào Yjs document
5. Yjs đồng bộ update tới các client khác qua WebSocket
6. React Flow tự động render lại theo dữ liệu mới

Cách tổ chức này đảm bảo UI luôn phản ánh trạng thái dữ liệu đã được đồng bộ, tránh tình trạng sai lệch giữa các client.

2.4. Real-time Collaboration với WebSocket và Yjs

Một trong những yêu cầu quan trọng của ứng dụng Mindmap hiện đại là khả năng cộng tác thời gian thực (real-time collaboration), cho phép nhiều người dùng cùng chỉnh sửa một sơ đồ tư duy và quan sát thay đổi ngay lập tức. Để đáp ứng yêu cầu này, hệ thống sử dụng **WebSocket (Y-websocket)** cho giao tiếp thời gian thực và **Yjs** – một framework CRDT – để đồng bộ dữ liệu phân tán một cách nhất quán và không xung đột.

2.4.1. WebSocket Protocol

WebSocket là giao thức giao tiếp 2 chiều **full-duplex** hoạt động trên một kết nối TCP duy nhất. Khác với mô hình HTTP request–response, WebSocket cho phép **server chủ động gửi dữ liệu xuống client** ngay khi có thay đổi, phù hợp với các ứng dụng cộng tác thời gian thực.

Quá trình thiết lập kết nối (Handshake)

Quá trình handshake của WebSocket gồm các bước:

1. Client gửi HTTP request với header Upgrade: websocket
2. Server chấp nhận và nâng cấp kết nối lên WebSocket protocol
3. Kết nối WebSocket được thiết lập, cho phép gửi và nhận dữ liệu hai chiều liên tục

Ưu điểm của WebSocket

- **Low latency:** Giảm độ trễ do không cần tạo lại kết nối
- **Real-time:** Dữ liệu được push ngay khi có thay đổi
- **Efficient:** Ít overhead hơn so với HTTP long-polling

WebSocket trong dự án

Trong dự án, thư viện **y-websocket** được sử dụng cho việc giao tiếp WebSocket, đảm nhiệm:

- Quản lý kết nối và vòng đời session
- Tự động reconnect khi mất kết nối
- Gom (batch) các message để tối ưu băng thông

Server WebSocket chủ yếu đóng vai trò chuyển tiếp các update giữa các client.

2.4.2. Yjs – CRDT Framework

Yjs là thư viện JavaScript triển khai **CRDT (Conflict-free Replicated Data Type)**, cho phép nhiều client chỉnh sửa dữ liệu đồng thời mà không xảy ra xung đột, đồng thời đảm bảo **eventual consistency (tính nhất quán cuối cùng)** trong hệ thống phân tán.

Cơ chế hoạt động

Cách thức đồng bộ của Yjs trong hệ thống như sau:

1. Mỗi client giữ một bản sao local của tài liệu (Y.Doc).
2. Khi người dùng thực hiện thay đổi, Yjs tạo ra một update.
3. Update được gửi qua WebSocket đến các client khác.
4. Mỗi client apply update vào Y.Doc local.
5. Yjs tự động merge các thay đổi mà không cần khóa hay điều phối trung tâm

Kiểu dữ liệu sử dụng

Trong ứng dụng Mindmap, Yjs quản lý dữ liệu chính gồm:

```
const ydoc = new Y.Doc()
const yNodes = ydoc.getMap('nodes')
const yEdges = ydoc.getArray('edges')
```

- **Y.Map:** Lưu trữ dữ liệu node theo dạng key-value
- **Y.Array:** Lưu trữ danh sách edge có thứ tự

Lưu trữ và khôi phục dữ liệu

Để đảm bảo tính bền vững (persistence):

- Trạng thái Y.Doc được mã hóa thành binary update
- Update được lưu vào database (MongoDB)
- Khi tải lại tài liệu, các update được apply lên Y.Doc rồi để khôi phục trạng thái

Undo / Redo

Yjs cung cấp **UndoManager** để theo dõi lịch sử chỉnh sửa:

```
const undoManager = new Y.UndoManager([yNodes, yEdges], {
  captureTimeout: 500
})
```

2.5. OpenAI GPT Model

GPT (Generative Pre-trained Transformer) là họ models được train trên massive text data. GPT-4 và GPT-4o-mini (sử dụng trong dự án) có khả năng:

- Natural Language Understanding: Hiểu context, intent
- Text Generation: Sinh văn bản coherent

- Few-shot Learning: Học từ vài ví dụ trong prompt
- Reasoning: Suy luận logic

Trong dự án này, AI được gọi qua API để phân tích văn bản đầu vào và sinh cấu trúc mindmap dạng JSON. Ngoài ra, AI còn gợi ý node mới dựa trên ngữ cảnh của sơ đồ hiện tại.

2.6. Cơ sở dữ liệu

MongoDB là hệ quản trị cơ sở dữ liệu NoSQL dạng document, lưu trữ dữ liệu dưới định dạng BSON. Khác với cơ sở dữ liệu quan hệ, MongoDB không yêu cầu schema cố định, phù hợp với các ứng dụng có cấu trúc dữ liệu linh hoạt và thường xuyên thay đổi.

Trong các hệ thống web hiện đại, MongoDB thường được sử dụng để lưu trữ dữ liệu ứng dụng như thông tin người dùng, nội dung tài liệu và trạng thái hệ thống. Đặc điểm này đặc biệt phù hợp với các ứng dụng cộng tác thời gian thực, nơi dữ liệu có thể được cập nhật liên tục.

Mongoose là một Object Data Modeling (ODM) cho MongoDB trong môi trường Node.js. Mongoose cung cấp cơ chế ánh xạ giữa object trong chương trình và document trong MongoDB, giúp:

- Định nghĩa cấu trúc dữ liệu một cách rõ ràng
- Hỗ trợ validation và middleware
- Giảm độ phức tạp khi thao tác với cơ sở dữ liệu

Việc sử dụng MongoDB kết hợp với Mongoose giúp hệ thống quản lý dữ liệu hiệu quả và dễ bảo trì.

Cơ sở dữ liệu sẽ lưu các thông tin như người dùng, các phiên bản (version), các mindmaps.

Chương 3 - Phân tích và thiết kế hệ thống

3.1. Phân tích yêu cầu hệ thống

3.1.1. Yêu cầu chức năng (Functional Requirements)

FR-01: Quản lý tài khoản người dùng

- FR-01.1: Đăng ký tài khoản mới với email và password
- FR-01.2: Đăng nhập với email/password, nhận JWT tokens
- FR-01.3: Làm mới access token bằng refresh token
- FR-01.4: Xem và cập nhật profile (name, avatar)
- FR-01.5: Đổi mật khẩu với xác thực password cũ
- FR-01.6: Upload và hiển thị avatar (hỗ trợ JPG, PNG, tối đa 5MB)

FR-02: Quản lý Mindmap

- FR-02.1: Tạo mindmap mới với title và description
- FR-02.2: Xem danh sách mindmaps (owned và shared)
- FR-02.3: Xem chi tiết một mindmap
- FR-02.4: Xóa mindmap (chỉ owner, tạo backup trước khi xóa)
- FR-02.5: Phân biệt owned và shared mindmaps trong dashboard

FR-03: Vẽ và chỉnh sửa Mindmap

- FR-03.1: Thêm node mới (child node hoặc free node)
- FR-03.2: Chỉnh sửa text của node (double-click để edit)
- FR-03.3: Xóa node (Delete key, xóa cả subtree của node đó)
- FR-03.4: Di chuyển node (drag & drop)
- FR-03.5: Reparenting node (kéo node lên node khác để thay đổi node cha)
- FR-03.6: Tùy chỉnh style node: màu nền, màu chữ, font size, bold/italic/underline
- FR-03.7: Tạo custom connection giữa 2 nodes bất kỳ
- FR-03.8: Tùy chỉnh custom connection: màu, độ dày, kiểu line (solid/dashed/dotted)
- FR-03.9: Auto-layout: Tự động sắp xếp nodes theo cây cân bằng

FR-04: Real-time Collaboration

- FR-04.1: Nhiều users chỉnh sửa đồng thời
- FR-04.2: Đồng bộ changes tức thì ($< 1s$ latency)
- FR-04.3: Hiển thị số lượng users online
- FR-04.4: Xử lý conflicts tự động với CRDT

FR-05: Version Control

- FR-05.1: Lưu version thủ công với label tùy chỉnh
- FR-05.2: Xem danh sách versions (sorted by time desc)
- FR-05.3: Restore về version cũ
- FR-05.4: Tạo backup tự động trước khi restore
- FR-05.5: Hiển thị metadata: người tạo, thời gian, kích thước, type (auto/manual/restore)

FR-06: Quản lý cộng tác (Collab Management)

- FR-06.1: Owner mời collaborators qua email
- FR-06.2: Chọn role khi mời (editor hoặc viewer)
- FR-06.3: Gửi notification đến người được mời
- FR-06.4: Người được mời accept/reject invitation
- FR-06.5: Xem danh sách collaborators với roles
- FR-06.6: Owner thay đổi role của collaborator
- FR-06.7: Owner xóa collaborator
- FR-06.8: Viewer mode: Read-only, không sửa được

FR-07: Thông báo (Notification)

- FR-07.1: Hiển thị notification bell với unread count
- FR-07.2: Dropdown list notifications (sorted by time)
- FR-07.3: Accept collaboration invite → Được thêm vào collaborators
- FR-07.4: Reject collaboration invite → Thông báo đổi status

- FR-07.5: Mark notification as read

FR-08: AI

- FR-08.1: Generate mindmap từ text prompt
- FR-08.2: AI trả về cấu trúc JSON hierarchical (root + children)
- FR-08.3: Tự động apply structure vào canvas
- FR-08.4: Auto-layout sau khi generate
- FR-08.5: AI suggest child nodes dựa trên context (current node, parents, siblings)
- FR-08.6: Hiển thị 3-5 suggestions
- FR-08.7: Click suggestion để add node
- FR-08.8: Hỗ trợ tiếng Việt và tiếng Anh

FR-09: Keyboard Shortcuts & UX

- FR-09.1: Tab: Thêm child node
- FR-09.2: Enter: Thêm sibling node
- FR-09.3: Delete/Backspace: Xóa node
- FR-09.4: Ctrl+Z: Undo
- FR-09.5: Ctrl+Y: Redo
- FR-09.6: Escape: Dừng connection mode
- FR-09.7: Double-click: Chỉnh text của node
- FR-09.8: Loading indicators khi xử lý bất đồng bộ

3.1.2. Yêu cầu phi chức năng (Non-Functional Requirements)

NFR-01: Performance

- Real-time sync latency < 1 giây
- API response time < 2 giây (bình thường)
- AI generation time < 10 giây
- Auto-layout calculation < 500ms cho mindmap 100 nodes
- Frontend bundle size < 1MB (gzipped)

NFR-02: Scalability

- Hỗ trợ nhiều concurrent users trên 1 mindmap
- Database có thể lưu trữ nhiều mindmaps
- Realtime server xử lý nhiều WebSocket connections

NFR-03: Security

- Password hashed với bcrypt (10 rounds)
- JWT tokens signed với secret key
- Refresh tokens có expiry và lưu trong database
- API endpoints protected với authentication middleware
- Role-based access control (owner/editor/viewer)
- Service-to-service communication với service tokens
- Input validation để tránh injection attacks

NFR-04: Usability

- Tuân thủ Material Design principles
- Error messages rõ ràng, user-friendly
- Confirmation dialogs cho actions quan trọng (delete, restore)
- Keyboard shortcuts cho người dùng
- Tooltips giải thích chức năng

NFR-05: Reliability

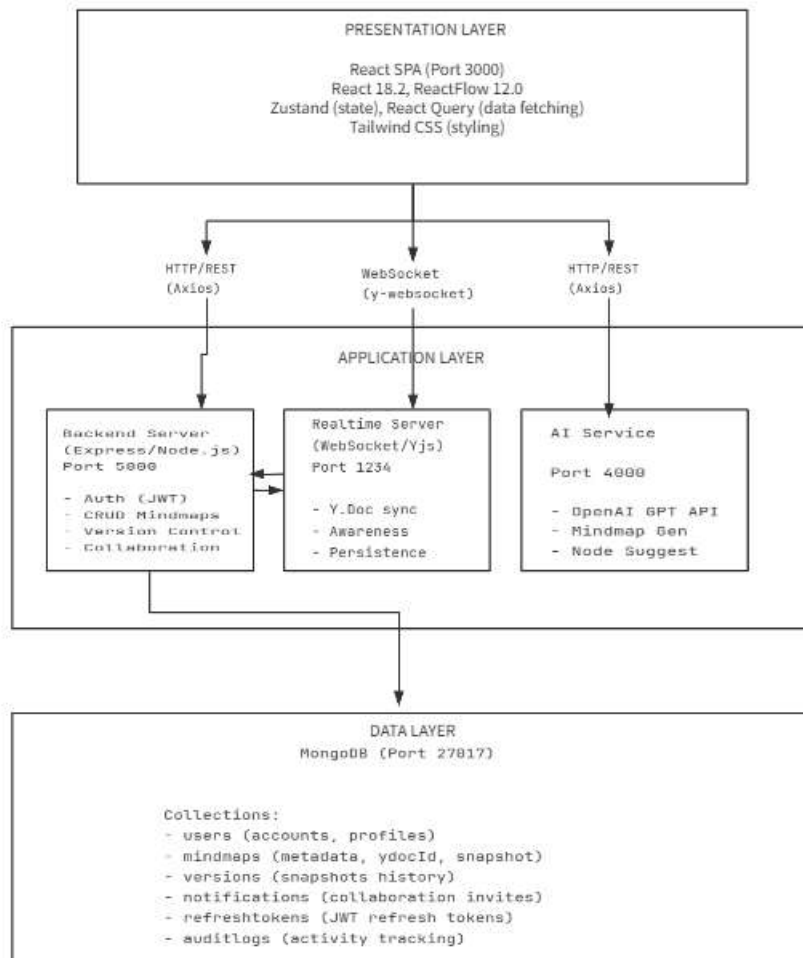
- Auto-reconnect khi WebSocket connection bị mất
- Transaction rollback khi có lỗi
- Backup tự động trước khi restore version

NFR-06: Maintainability

- Code structure backend rõ ràng (MVC pattern)
- Comments cho logic phức tạp
- Tách biệt logic với UI

3.2. Kiến trúc hệ thống

3.2.1. Kiến trúc tổng thể



3.2.2. Luồng xử lý chính

3.2.2.1. User Authentication Flow:

User → Frontend → POST /api/auth/login (email, password)

→ Backend: Xác thực

→ Tạo access token (15min) + refresh token (7 days)

→ Trả về tokens + user info

→ Frontend: Lưu tokens in Zustand + localStorage

→ Thêm access token vào các API requests (Authorization header)

3.2.2.2. Create Mindmap Flow:

User → Frontend → POST /api/mindmaps (title, description)

- Backend: Tạo Mindmap doc with unique ydocId
- Lưu vào MongoDB
- Trả mindmap object
- Frontend: Định hướng tới /mindmap/:id
- Kết nối Realtime Server qua Y-WebSocket
- Load Y.Doc mới (mindmap mới)
- Tạo root node

3.2.2.3. Real-time Edit Flow:

User A sửa node

- Frontend: cập nhật Y.Doc ở local
- Yjs tạo update message
- WebSocket gửi update tới Realtime Server
- Server truyền (broadcasts) tới các clients đang kết nối
- User B nhận update
- Frontend B cập nhật update vào Y.Doc
- ReactFlow render lại với trạng thái node mới

3.2.2.4. Save Version Flow:

User → Frontend → Bấm "Save Version"

- POST /api/mindmaps/:id/versions/save (label)
- Backend → GET /api/internal/mindmaps/:ydocId/snapshot (tới Realtime)
- Realtime: Mã hóa Y.Doc thành Uint8Array
- Trả về snapshot (base64 encodedState)
- Backend: Tạo Version document
- Cập nhật Mindmap.snapshot
- Trả về success

3.2.2.5. Restore Version Flow:

User → Frontend → Chọn version, bấm "Restore"

- POST /api/mindmaps/:id/versions/:versionId/restore
- Backend: Load version snapshot

- Tạo backup trạng thái hiện tại
- POST /broadcast-restore tới Realtime Server
- Realtime: cập nhật snapshot to Y.Doc
- Broadcast đến tất cả client
- Frontend: nhận update, render lại

3.2.2.6. AI Generate Mindmap Flow:

User → Frontend → Viết prompt, bấm "Generate"

- POST /ai/generate-mindmap (text prompt)
- AI Service: gọi OpenAI GPT API
- GPT Trả về response dạng json
- Trả về mindmap structure
- Frontend: Dùng structure đó vào Y.Doc
 - Xóa các node hiện tại
 - Tạo Root node với text mới
 - Tạo các node con
 - Auto-layout tự căn chỉnh.

3.2.3. Giao tiếp giữa các thành phần

Client-Backend:

- Protocol: HTTP/REST
- Format: JSON
- Auth: JWT ở Authorization header
- Error handling: HTTP status codes + error messages

Client-Realtime:

- Protocol: WebSocket (upgraded from HTTP)
- Auth: Token trong query param (ws://server?token=...)
- Message format: Yjs binary updates
- Reconnection: tự động reconnect với exponential backoff

Backend-Realtime:

- Protocol: HTTP

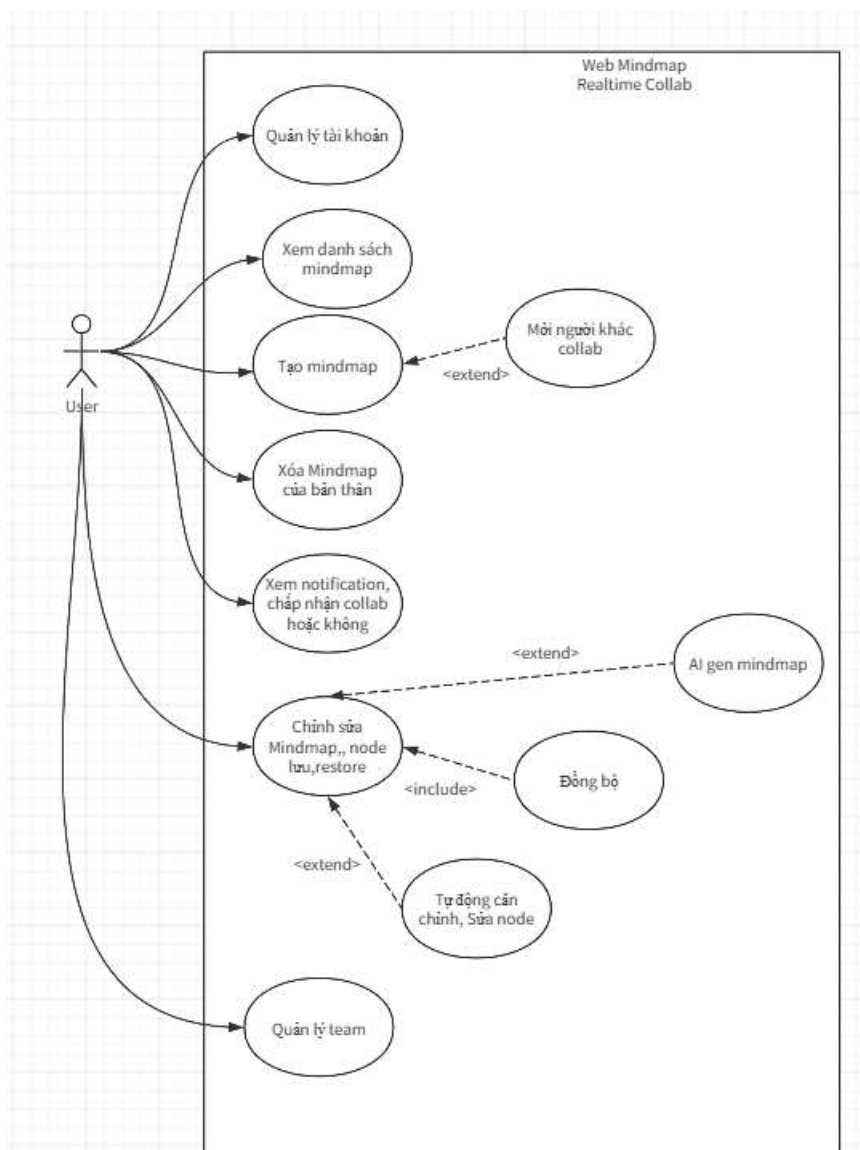
- Auth: Service token in x-service-token header
- Purpose: Trigger snapshot save, verify user access

Frontend-AI Service:

- Protocol: HTTP
- Auth: Không (internal service)
- Tạo mindmap, gợi ý nodes

3.3. Thiết kế Use Case

Sơ bộ về usecase với factor duy nhất của hệ thống là User.



MÔ TẢ CHI TIẾT CÁC USE CASE

3.3.1. Danh sách các Use Case chính

- **UC-01:** Đăng ký tài khoản
- **UC-02:** Đăng nhập
- **UC-03:** Quản lý profile
- **UC-04:** Tạo mindmap mới
- **UC-05:** Xem danh sách mindmaps
- **UC-06:** Vẽ sơ đồ tư duy
- **UC-07:** Real-time collaboration
- **UC-08:** Lưu version
- **UC-09:** Xem version history
- **UC-10:** Restore version
- **UC-11:** Mời collaborators
- **UC-12:** Quản lý team members
- **UC-13:** Accept/Reject invite
- **UC-14:** Generate mindmap với AI
- **UC-15:** AI suggest nodes

Actor: Người dùng (User)

3.3.2. Use Case Chi tiết

UC-01: Đăng ký tài khoản

Primary Actor: Người dùng

Preconditions:

- User chưa có tài khoản trong hệ thống
- User có email.

Main Flow:

1. User truy cập trang đăng ký
2. Hệ thống hiển thị form đăng ký (email, password, name)

3. User nhập thông tin:
 - 3a. Email
 - 3b. Password (tối thiểu 6 ký tự)
 - 3c. Full name
4. User click button "Sign Up"
5. Hệ thống validate thông tin:
 - Email format hợp lệ
 - Email chưa tồn tại trong database
6. Hệ thống hash password với bcrypt
7. Hệ thống tạo User document trong MongoDB
8. Hệ thống hiển thị success message
9. Redirect đến trang login

Alternative Flows:

- 5a. Email đã tồn tại:
 - Hiển thị error "Email already exists"
 - Quay lại bước 3
- 5b. Password không đủ mạnh:
 - Hiển thị error "Password must be at least 6 characters"
 - Quay lại bước 3
- 5c. Email format không hợp lệ:
 - Hiển thị error "Invalid email format"
 - Quay lại bước 3

Postconditions:

- User account được tạo trong database
- Password được hash an toàn
- User có thể đăng nhập với credentials vừa tạo

Primary Actor: Người dùng

Preconditions:

- User đã có tài khoản
- User chưa đăng nhập

Main Flow:

1. User truy cập trang login
2. Hệ thống hiển thị form login (email, password)
3. User nhập email và password
4. User click button "Login"
5. Hệ thống gọi API POST /auth/login
6. Backend verify credentials:
 - Tìm user theo email
 - So sánh password với bcrypt
7. Backend generate tokens:
 - Access token (JWT, 15 phút)
 - Refresh token (JWT, 7 ngày)
8. Backend trả về response: {accessToken, refreshToken, user}
9. Frontend lưu tokens vào:
 - localStorage (persistence)
 - Zustand store (state management)
10. Frontend set Authorization header: Bearer {accessToken}
11. Redirect đến dashboard

Alternative Flows:

- 6a. Email không tồn tại:
 - Trả về 401 "Invalid credentials"
 - Hiển thị error message
 - Quay lại bước 3
- 6b. Password sai:

- Trả về 401 "Invalid credentials"
 - Hiển thị error message
 - Quay lại bước 3
- 7a. Token generation fails:
 - Trả về 500 "Login failed"
 - Log error
 - Quay lại bước 3

Postconditions:

- User được authenticate
- Access token và refresh token được lưu
- Tất cả API requests có Authorization header
- User có thể truy cập các chức năng.

UC-03: Quản lý profile

Primary Actor: Người dùng

Preconditions:

- User đã đăng nhập
- Access token còn hợp lệ

Main Flow:

1. User click vào avatar/name trên header
2. Hệ thống hiển thị dropdown menu
3. User chọn "Profile Settings"
4. Hệ thống load user profile từ API GET /users/me
5. Hiển thị form với current data:
 - Name
 - Email (read-only)
 - Avatar URL
6. User chỉnh sửa thông tin:

- 6a. Update name
- 6b. Upload avatar mới
- 7. User click "Save Changes"
- 8. Hệ thống validate input
- 9. Gọi API PATCH /users/me với updated data
- 10. Backend cập nhật User document
- 11. Backend trả về updated user object
- 12. Frontend update Zustand store
- 13. Hiển thị success toast "Profile updated"
- 14. UI re-render với data mới

Alternative Flows:

- 6b1. Upload avatar:
 - User click "Change Avatar"
 - Mở file picker
 - User chọn image file
 - Frontend upload lên cloud storage (Cloudinary)
 - Nhận về avatar URL
 - Update preview
- 8a. Validation fails:
 - Hiển thị error messages
 - Quay lại bước 6
- 9a. API error:
 - Hiển thị error toast
 - Không đóng modal
 - User có thể retry

Postconditions:

- User profile được cập nhật
- Avatar mới hiển thị trên UI

- Changes được lưu vào database
-

UC-04: Tạo mindmap mới

Primary Actor: Người dùng

Preconditions:

- User đã đăng nhập
- User có quyền tạo mindmap

Main Flow:

1. User click button "New Mindmap" trên dashboard
2. Hệ thống hiển thị modal create mindmap
3. Hiển thị form với fields:
 - Title (required)
 - Description (optional)
4. User nhập title và description
5. User click "Create"
6. Frontend gọi API POST /mindmaps với data
7. Backend tạo Mindmap document:
 - Generate unique ydocId (UUID)
 - Set ownerId = current user
 - Initialize snapshot = null
 - Set createdAt, updatedAt
8. Backend lưu vào MongoDB
9. Backend trả về mindmap object: {id, title, description, ydocId}
10. Frontend navigate đến /mindmap/:id
11. Frontend establish WebSocket connection:
 - URL: ws://realtime-server?ydocId={ydocId}&userId={userId}
12. Realtime Server verify connection và create/load Y.Doc
13. Server gửi initial state (empty doc)

14. Frontend initialize Y.Doc và bind với ReactFlow

15. Frontend tạo root node tự động:

- Label = title
- Position = center
- Type = "root"

16. Hiển thị empty canvas với root node

Alternative Flows:

- 4a. User không nhập description:
 - Description = null
 - Continue flow
- 6a. Title đã tồn tại cho user này:
 - Backend auto append số (e.g., "Title (2)")
- 11a. WebSocket connection fails:
 - Hiển thị error "Connection failed"
 - Retry sau 3 giây (max 5 lần)
 - Nếu fail hết → Redirect về dashboard

Postconditions:

- Mindmap document được tạo trong database
- Y.Doc được initialize trên Realtime Server
- User được chuyển đến editor
- WebSocket connection active
- Root node hiển thị trên canvas

UC-05: Xem danh sách mindmaps

Primary Actor: Người dùng

Preconditions:

- User đã đăng nhập

Main Flow:

1. User truy cập trang dashboard
2. Hệ thống gọi API GET /mindmaps
3. Backend query MongoDB:
 - Tìm mindmaps where ownerId = userId
 - Hoặc user trong collaborators array
4. Backend trả về array of mindmaps sorted by updatedAt desc
5. Frontend hiển thị grid/list view với mỗi mindmap:
 - Thumbnail (snapshot preview)
 - Title
 - Last modified time
 - Owner name
 - Collaborators count
6. User có thể:
 - 6a. Click vào mindmap → Open (đến UC-06)
 - 6b. Click nút delete ở MindmapCard ở Dashboard → Delete mindmap của bản thân (đến 9)

Main Flow - Delete (9): 9. User click icon delete trên mindmap card

10. Hiển thị confirmation dialog
11. User confirm
12. Gọi API DELETE /mindmaps/:id
13. Backend xóa Mindmap document và related data
14. Remove khỏi list
15. Hiển thị success toast

Alternative Flows:

- 2a. No mindmaps:
 - Hiển thị empty state với CTA "Create your first mindmap"
- 12a. User không phải owner: Không có nút xóa Mindmap, cũng không xóa được do RBAC.

Postconditions:

- Danh sách mindmaps được hiển thị
 - User có thể xem, xóa mindmaps
-

UC-06: Vẽ sơ đồ tư duy

Primary Actor: Người dùng

Preconditions:

- User đã đăng nhập
- User có quyền edit (owner hoặc editor)
- Mindmap đã được load

Main Flow:

1. User click vào một node
2. Hệ thống hiển thị floating toolbar với options
3. User chọn thao tác:
 - 3a. Click nút "+" → Thêm child node (đến 4)
 - 3b. Double-click node → Edit text (đến 5)
 - 3c. Chọn màu sắc → Update node color (đến 6)
 - 3d. Click nút connection → Bật connection mode (đến 7)
 - 3e. Press Delete → Xóa node (đến 8)

Sub-flow 4 - Thêm child node:

4. Hệ thống tạo node mới với position tính toán
5. Node được thêm vào Y.Doc: `yNodes.set(newId, nodeData)`
6. Auto-layout được trigger (dagre algorithm)
7. Focus vào node mới để nhập text
8. Yjs sync update đến all clients

Sub-flow 5 - Edit text:

5. Hệ thống chuyển node sang edit mode
6. User gõ text mới
7. Press Enter hoặc click ra ngoài:

- Lưu text vào Y.Doc
- Exit edit mode

8. Press Escape:

- Cancel edit
- Restore original text

Sub-flow 6 - Update color:

6. Hệ thống hiển thị color picker

7. User chọn màu từ palette

8. Node color được update: `yNodes.get(nodeId).color = newColor`

9. ReactFlow re-render node với màu mới

Sub-flow 7 - Connection mode:

7. Hệ thống hiển thị anchors (top/right/bottom/left) trên tất cả nodes

8. User click anchor trên node A (source)

9. User click anchor trên node B (target)

10. Hệ thống tạo custom edge từ $A \rightarrow B$

11. Edge được thêm vào Y.Edges: `yEdges.set(edgeId, edgeData)`

Sub-flow 8 - Xóa node:

8. Hệ thống hiển thị confirmation dialog

9. User confirm

10. Recursively collect node và tất cả children IDs

11. Xóa tất cả nodes khỏi Y.Doc

12. Xóa tất cả edges liên quan

13. ReactFlow re-render canvas

Alternative Flows:

- 3f. Drag node → Di chuyển node, có thể reparent
- 3g. Press Tab → Thêm child node (keyboard shortcut)
- 3h. Click AI suggest → Hiển thị AI suggestions

Postconditions:

- Mindmap được cập nhật

- Changes được đồng bộ đến all clients trong < 1 giây
- Auto-save timer được reset

UC-07: Real-time collaboration

Primary Actor: Người dùng, Other Users

Preconditions:

- Nhiều users đã mở cùng một mindmap
- Tất cả WebSocket connections active
- Y.Doc đã được load

Main Flow:

1. User A edit node "Marketing"
2. Frontend A update Y.Doc locally: `yNodes.get(nodeId).label = "Marketing Strategy"`
3. Yjs tạo update message (binary delta)
4. WebSocket gửi update đến Realtime Server
5. Realtime Server nhận update
6. Server broadcast update đến tất cả connected clients (User B, C, D)
7. User B, C, D nhận update qua WebSocket
8. Frontend B, C, D apply update vào Y.Doc
9. Yjs merge update (CRDT algorithm)
10. ReactFlow re-render với new state
11. User B, C, D thấy node updated sau < 1 giây

Sub-flow - Awareness (Cursors & Online Users):

1. Frontend update awareness state: `awareness.setLocalState({user: {...}, cursor: {x, y}})`
2. Yjs broadcast awareness qua WebSocket
3. Other users receive awareness update
4. Hiện thị avatar của User A trong "Active Users" list

Alternative Flows:

- 4a. Network chậm:

- Update được queue trong Yjs
 - Khi network OK, auto sync missing updates
- 7a. User B disconnected:
 - Changes được lưu local trong Y.Doc
 - Khi reconnect, Yjs auto sync state
- 9a. Conflict (2 users edit cùng lúc):
 - Yjs CRDT tự động merge without data loss
 - Last write wins cho simple fields

Postconditions:

- Tất cả users thấy same state
- No data loss
- Conflicts được resolve tự động
- Latency < 1 giây

UC-08: Lưu version

Primary Actor: Người dùng (Owner hoặc Editor)

Preconditions:

- User đã đăng nhập
- User có quyền write
- Mindmap đã có thay đổi

Main Flow:

1. User click button "Save Version" trên toolbar
2. Hệ thống hiển thị modal với input field "Version Label"
3. User nhập label (optional, default = "Version {date}")
4. User click "Save"
5. Frontend gọi API POST /mindmaps/:id/versions/save với {label}
6. Backend gọi internal API GET /internal/mindmaps/:ydocId/snapshot
7. Realtime Server:

- Encode current Y.Doc state thành Uint8Array
 - Convert sang base64 string
- 8. Realtime Server trả về {snapshot: base64String}
- 9. Backend tạo Version document:
- 10. { mindmapId: ObjectId, createdBy: userId, label: "v1.0", snapshot: base64String, createdAt: new Date() }
- 11. Backend lưu Version vào MongoDB
- 12. Backend update Mindmap.snapshot = latest snapshot
- 13. Backend trả về version object
- 14. Frontend hiển thị success toast "Version saved"
- 15. Modal đóng

Alternative Flows:

- 3a. User không nhập label:
 - Auto generate: "Version {MM/DD/YYYY HH:mm}"
- 6a. Snapshot encoding fails:
 - Trả về 500 error
 - Hiển thị "Failed to save version"
 - User có thể retry

Postconditions:

- Version được lưu trong database
- Snapshot có thể restore về sau
- Version hiển thị trong history
- Current snapshot được backup

UC-09: Xem version history

Primary Actor: Người dùng

Preconditions:

- User đã đăng nhập
- User có quyền read

- Mindmap có ít nhất 1 saved version

Main Flow:

1. User click button "History" trên toolbar
2. Hệ thống mở Version History Modal (sidebar)
3. Frontend gọi API GET /mindmaps/:id/versions
4. Backend query MongoDB:
 - Tìm versions có mindmapId = current mindmap
 - Sắp xếp theo createdAt
5. Backend trả về array of versions: [{id, label, createdBy, createdAt}, ...]
6. Frontend hiển thị list với mỗi version:
 - Label
 - Creator name
 - Created timestamp (relative time)
7. User có thể:
 - 7a. Click "View" → Xem meta data của các Version
 - 7b. Click "Restore" → Restore version (đến UC-10)
8. User click "Close" để đóng modal

Alternative Flows:

- 3a. No versions saved:
 - Hiển thị empty state "No versions saved yet"

Postconditions:

- User xem được lịch sử versions
- User có thể chọn version để restore

UC-10: Restore version

Primary Actor: Người dùng (Owner hoặc Editor)

Preconditions:

- User đã đăng nhập

- User có quyền write
- Mindmap có ít nhất 2 versions

Main Flow:

1. User click button "History" trên toolbar
2. Hệ thống mở Version History Modal
3. Hệ thống load danh sách versions từ API
4. Hiển thị list versions sắp xếp theo thời gian
5. User select một version cũ
6. User click button "Restore"
7. Hệ thống hiển thị confirmation dialog với warning:
 - "This will replace current state with selected version"
 - "Current state will be backed up automatically"
8. User confirm restore
9. Hệ thống gọi API POST /mindmaps/:id/versions/:versionId/restore
10. Backend:
 - Load version snapshot từ MongoDB
 - Backup current state (create new version type="auto-backup")
 - POST /broadcast-restore to Realtime Server với snapshot
11. Realtime Server:
 - Decode base64 snapshot thành Uint8Array
 - Clear current Y.Doc state
 - Apply snapshot to Y.Doc
 - Broadcast restore event đến all connected clients
12. All clients receive update
13. Frontend re-render canvas với restored state
14. Hiển thị success toast "Version restored successfully"
15. Modal đóng

Alternative Flows:

- 8a. User cancel:
 - Đóng dialog
 - Không restore
 - Quay lại version list
- 9a. API lỗi (500):
 - Hiện thị error message "Failed to restore version"
 - Không đóng modal
 - User có thể retry
- 12a. Client disconnected:
 - Client reconnect
 - Yjs auto sync missing updates
 - Receive restored state khi online lại

Postconditions:

- Mindmap state giống version đã chọn
- Backup được tạo trong Version collection
- Tất cả users thấy updated state
- Version history có thêm 1 record type="restore"

UC-11: Mời collaborators

Primary Actor: Người dùng (Owner)

Preconditions:

- User là owner của mindmap
- User đã mở mindmap

Main Flow:

1. User click button "Teams" trên toolbar
2. Hệ thống hiển thị Share Modal
3. Hiện thị current collaborators list
4. User click "Add"

5. Hiển thị input field để nhập email
6. User nhập email của người cần mời
7. User chọn role (Viewer/Editor)
8. User click "Add"
9. Frontend validate email format
10. Gọi API POST /mindmaps/:id/invite với {email, role}
11. Backend:
 - Check user tồn tại với email
 - Tạo Notification document:
 - { mindmapId: ObjectId, invitedBy: userId, invitedEmail: email, role: "editor", status: "pending", createdAt: new Date() }
 - Gửi email notification đến invited user
12. Backend lưu Notification vào MongoDB
13. Trả về Notification object
14. Frontend thêm email vào "Pending Invites" list
15. Hiển thị success toast "Invitation sent to {email}"

Alternative Flows:

- 9a. Email format không hợp lệ:
 - Hiển thị error "Invalid email"
 - Quay lại bước 6
- 10a. User đã là collaborator:
 - Trả về 400 " Already a collaborator "
 - Hiển thị error
- 10b. Email không tồn tại trong hệ thống:
 - Trả về "User not found"
 - Gửi invite email với link đăng ký
 - Status = "pending_signup"
- 11a. Email service fails:
 - Invitation vẫn được tạo

- Log warning
- Continue flow

Postconditions:

- Invitation được tạo trong database
- Email notification được gửi
- Invited user nhận được thông báo

UC-12: Quản lý team members

Primary Actor: Người dùng (Owner)

Preconditions:

- User là owner của mindmap
- Mindmap có ít nhất 1 collaborator

Main Flow:

1. User click "Team" button
2. Hệ thống mở Share Modal
3. Frontend gọi API GET /mindmaps/:id/collaborators
4. Backend query:
 - Find mindmap và populate collaborators array
 - Include role và joinedAt
5. Backend trả về list: [{userId, name, email, role, joinedAt}, ...]
6. Frontend hiển thị table với columns:
 - Name, Email, Role.
7. User có thể:
 - 7a. Change role (đến 8)
 - 7b. Remove member (đến 9)

Sub-flow 8 - Change Role:

8. User click dropdown "Role" trên member row
9. Chọn new role (Viewer/Editor)

10. Gọi API PATCH /mindmaps/:id/collaborators/:userId với {role}
11. Backend update role trong Mindmap.collaborators array
12. Trả về updated collaborator
13. Frontend update UI
14. Hiển thị toast "Role updated"

Sub-flow 9 - Remove Member:

9. User click icon "Remove" trên member row
10. Hiển thị confirmation "Remove {name} from this mindmap?"
11. User confirm
12. Gọi API DELETE /mindmaps/:id/collaborators/:userId
13. Backend:
 - Remove userId từ collaborators array
 - User sẽ mất quyền truy cập mindmap
14. Frontend remove khỏi list
15. Hiển thị toast "{name} removed"

Alternative Flows:

- 10a. API fails:
 - Hiển thị error
 - Rollback UI change

Postconditions:

- Team members được quản lý
- Changes reflected trong database
- Removed users mất quyền truy cập

UC-13: Accept/Reject invite

Primary Actor: Người dùng (Invited User)

Preconditions:

- User đã nhận invitation email

- User đã đăng nhập

Main Flow:

1. User click link trong invitation email hoặc check notifications
2. Hệ thống redirect đến /invitations page
3. Frontend gọi API GET /invitations/me
4. Backend query Invitations where invitedEmail = user.email và status = "pending"
5. Backend trả về array of pending invitations
6. Frontend hiển thị list với mỗi invitation:
 - Mindmap title
 - Invited by (name)
 - Role
 - Invited date
 - Actions: Accept/Reject
7. User có thể:
 - 7a. Click "Accept" (đến 8)
 - 7b. Click "Reject" (đến 11)

Sub-flow 8 - Accept:

8. User click "Accept" button
 9. Frontend gọi API POST /invitations/:id/accept
 10. Backend:
 - Update Invitation.status = "accepted"
 - Add userId vào Mindmap.collaborators array với role
 - Tạo notification cho owner
 11. Trả về success
 12. Frontend remove invitation khỏi list
 13. Frontend redirect đến /mindmap/:id 14.
- Dashboard mindmap phần shared sẽ được cập nhật thêm mindmap được shared.

Sub-flow 11 - Reject:

11. User click "Reject" button
12. Frontend gọi API POST /invitations/:id/reject
13. Backend:
 - Update Invitation.status = "rejected"
 - Tạo notification cho owner
14. Frontend remove invitation khỏi list
15. Hiện thị ở phần notification là Declined

Alternative Flows:

- 4a. No pending invitations:
 - Hiện thị empty state "No pending invitations"
- 9a. Invitation đã expired:
 - Trả về 400 "Invitation expired"
 - Remove khỏi list

Postconditions:

- Invitation status được update
- Nếu accept: User được thêm vào collaborators
- Owner nhận notification

UC-14: Generate mindmap với AI

Primary Actor: Người dùng, AI System (OpenAI)

Preconditions:

- User đã đăng nhập
- User đang ở trong 1 mindmap
- AI API available

Main Flow:

1. User click button "Generate with AI"
2. Hệ thống hiển thị AI Generator Modal
3. Hiện thị textarea với placeholder "Describe your mindmap..."

4. User nhập prompt, ví dụ:
 - "Create a marketing strategy for a mobile fitness app"
5. User click "Generate"
6. Frontend gọi API POST /ai/generate-mindmap với {prompt}
7. AI Service:
 - Build system prompt:
 - You are a mindmap expert. Generate a structured mindmap in JSON format. Output must be valid JSON with this structure: { "root": "Main Topic", "children": [{ "label": "Subtopic 1", "children": [...] }, { "label": "Subtopic 2", "children": [...] }] }
 - Call OpenAI API (GPT-4) với prompt
8. OpenAI trả về JSON response
9. AI Service parse và validate JSON structure
10. Trả về mindmap structure cho Frontend
11. Frontend:
 - Clear existing nodes
 - Tạo root node với label từ AI
 - Recursively tạo children nodes:
 - Generate IDs
 - Calculate positions
 - Set colors
 - Tạo edges nối parent-child
 - Apply auto-layout (dagre algorithm)
12. Update Y.Doc với tất cả nodes và edges
13. Yjs sync với Realtime Server
14. ReactFlow render mindmap
15. Đóng modal và mindmap sẽ hiển thị.

Alternative Flows:

- 7a. OpenAI API fails (timeout/rate limit):

- Retry 1 lần
 - Nếu vẫn fail: Hiển thị error "AI service unavailable"
 - User có thể retry
- 8a. Invalid JSON response:
 - Parse error
 - Hiển thị "Invalid JSON"

Postconditions:

- Mindmap được tạo từ AI
- Structure logical và well-organized
- Tất cả nodes và edges được sync
- User có thể edit tiếp

UC-15: AI suggest nodes

Primary Actor: Người dùng, AI System

Preconditions:

- User đã mở mindmap
- User đang edit node
- AI API available

Main Flow:

1. User click chọn vào một node
2. Context menu hiển thị option "AI Suggest"
3. User click "AI Suggest"
4. Hệ thống hiển thị loading indicator trên node
5. Frontend collect context:
 - Current node label
 - Parent node label (if có)
 - Sibling nodes labels
6. Frontend gọi API POST /ai/suggest-nodes với:

7. { currentNode: "Marketing Strategy", parentNode: "Business Plan", siblings: ["Sales Strategy", "Operations"], count: 3}
8. AI Service:
 - Build prompt với context
 - Call OpenAI API
 - Request: "Suggest 3 child nodes for 'Marketing Strategy'"
9. OpenAI trả về suggestions: ["Social Media", "Content Marketing", "SEO Strategy"]
10. AI Service parse response
11. Trả về array of suggestions
12. Frontend hiển thị popover với list suggestions
13. User có thể:
 - 12a. Click suggestion → Add as child node (đến 13)
 - 12b. Click ra ngoài → Close popover
14. Frontend tạo child node với label = suggestion
15. Update Y.Doc
16. Yjs sync
17. Auto-layout adjust positions
18. Hiển thị new node với animation

Alternative Flows:

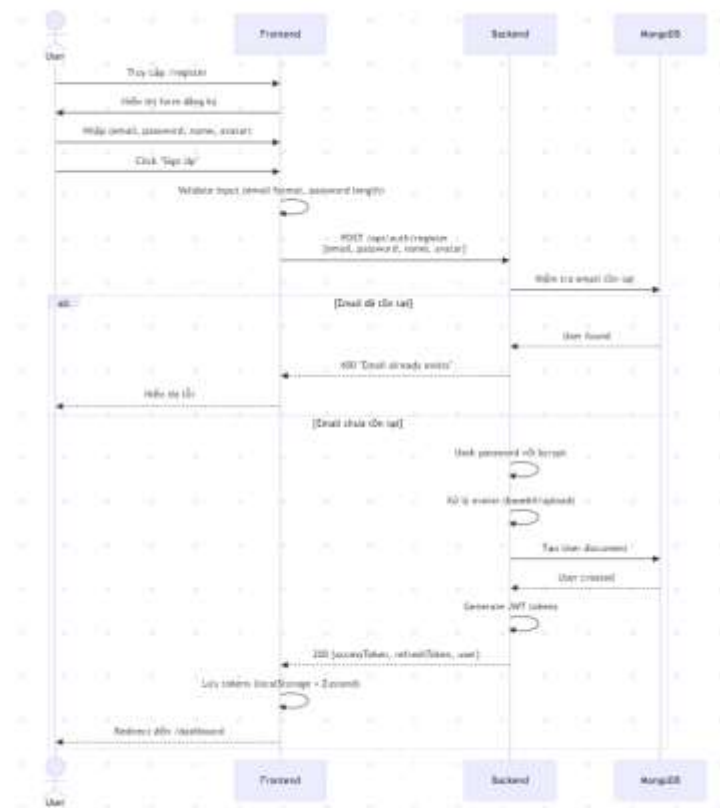
- 7a. AI returns < 3 suggestions:
 - Hiển thị available suggestions
 - No error
- 8a. API timeout:
 - Hiển thị "AI suggestions unavailable"
 - User có thể retry
- 12a. User chọn multiple suggestions:
 - Batch create nodes
 - Single Y.Doc update

Postconditions:

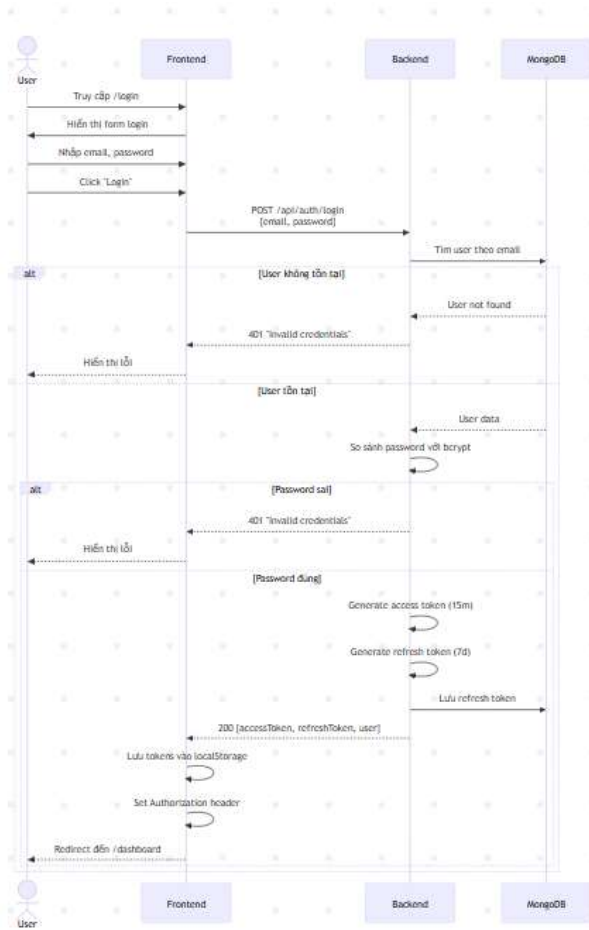
- AI suggestions được hiển thị
- User có thể nhanh chóng expand mindmap
- Suggestions liên quan với node.

3.4. Biểu đồ tuần tự (Sequence Diagrams)

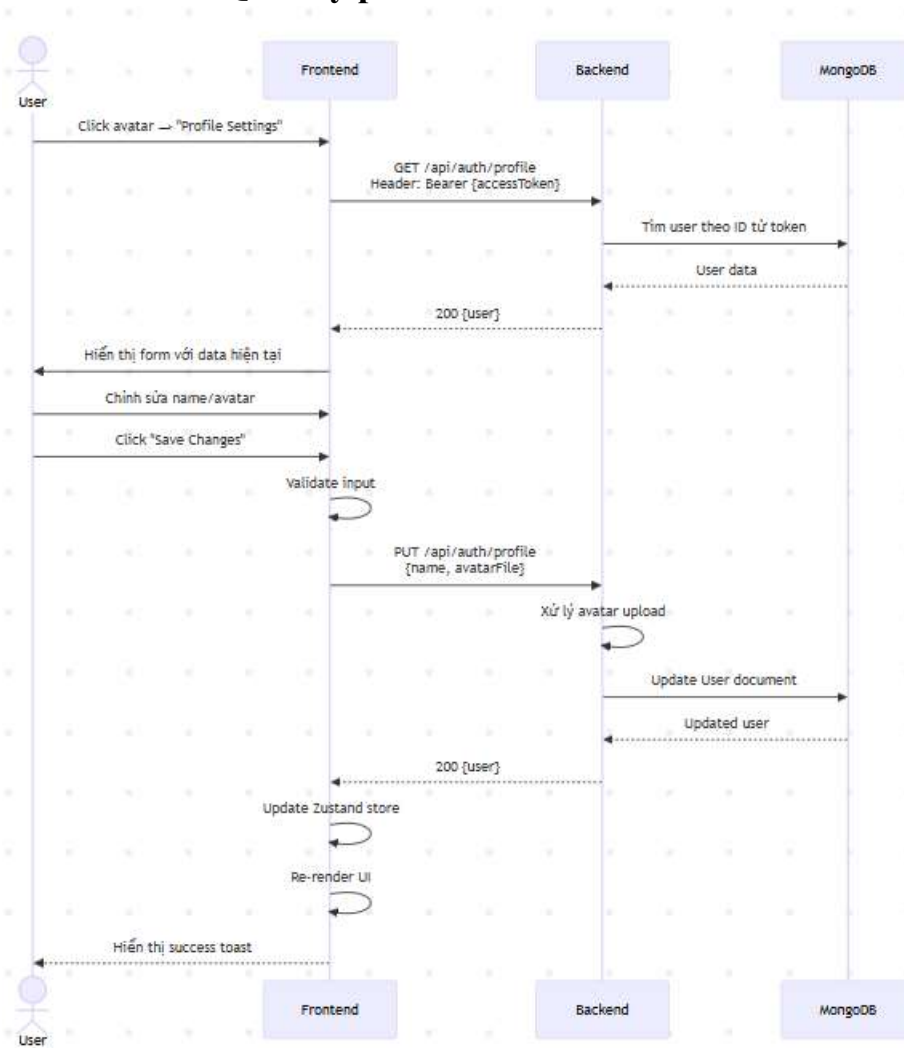
3.4.1. UC-01: Đăng ký tài khoản



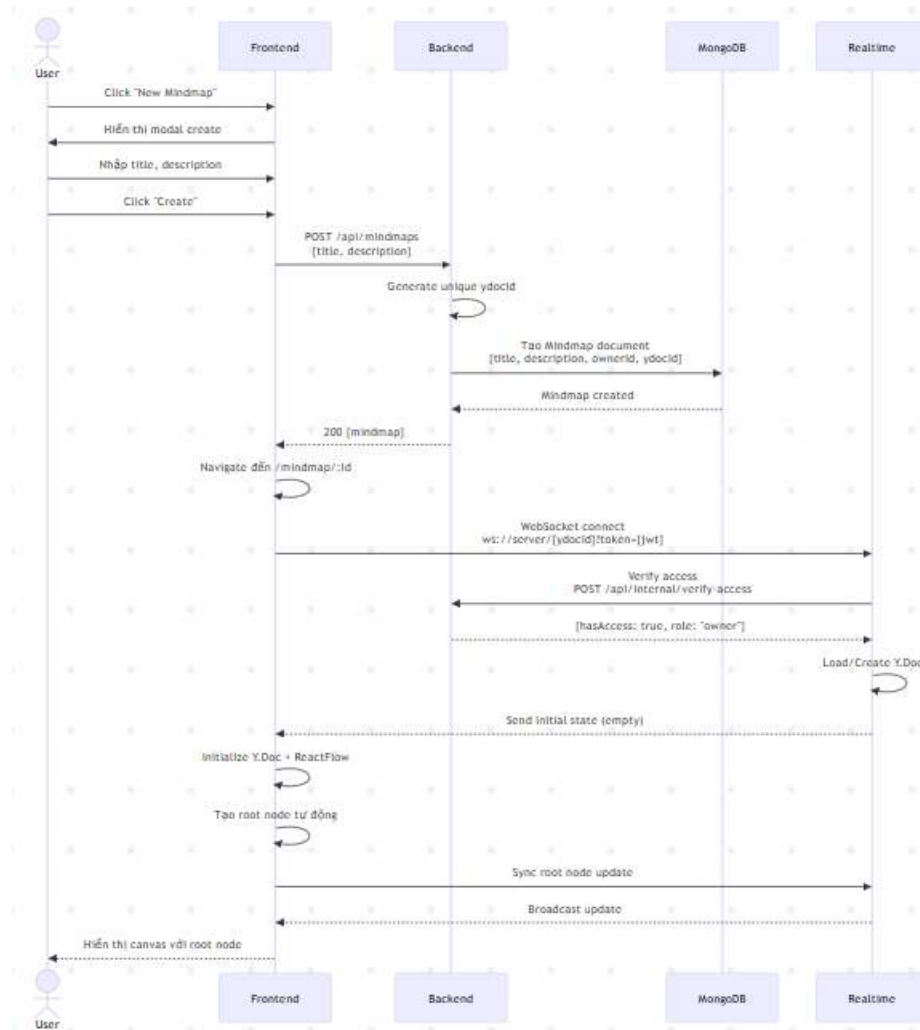
3.4.2. UC-02: Đăng nhập



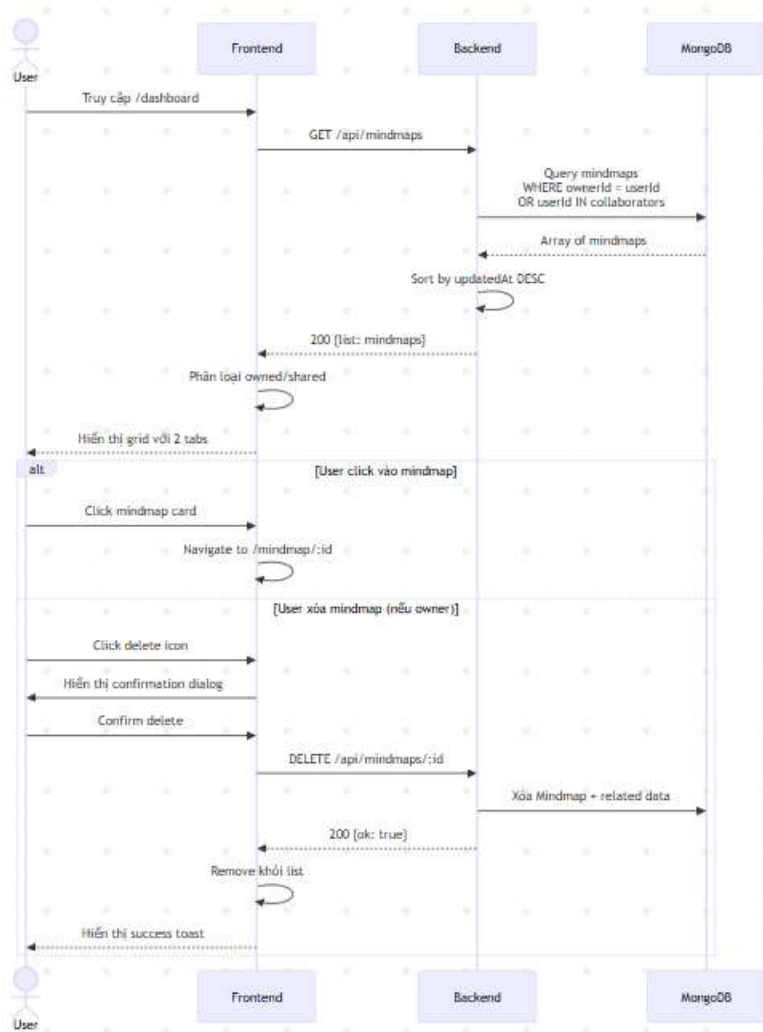
3.4.3. UC-03: Quản lý profile



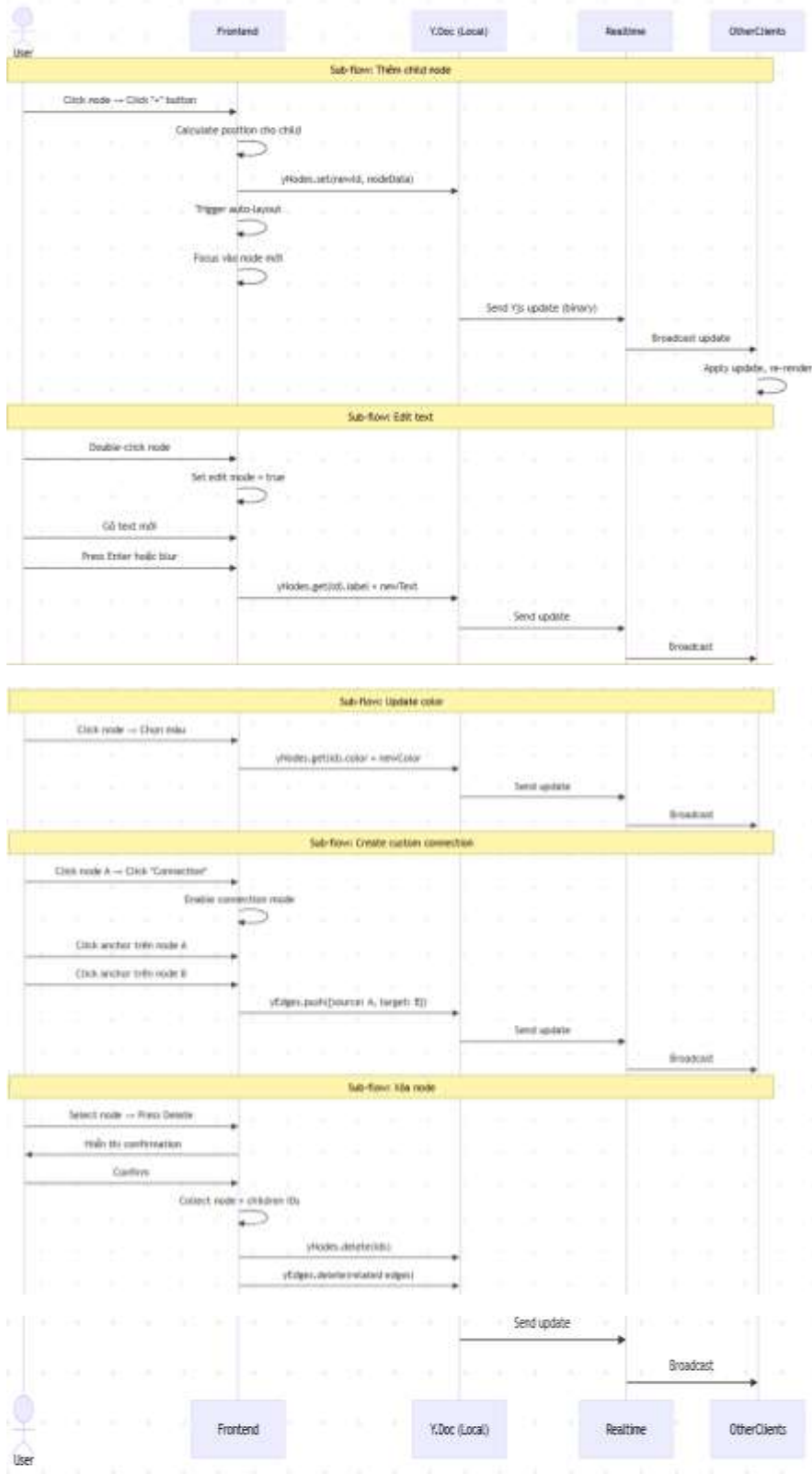
3.4.4. UC-04: Tạo mindmap mới



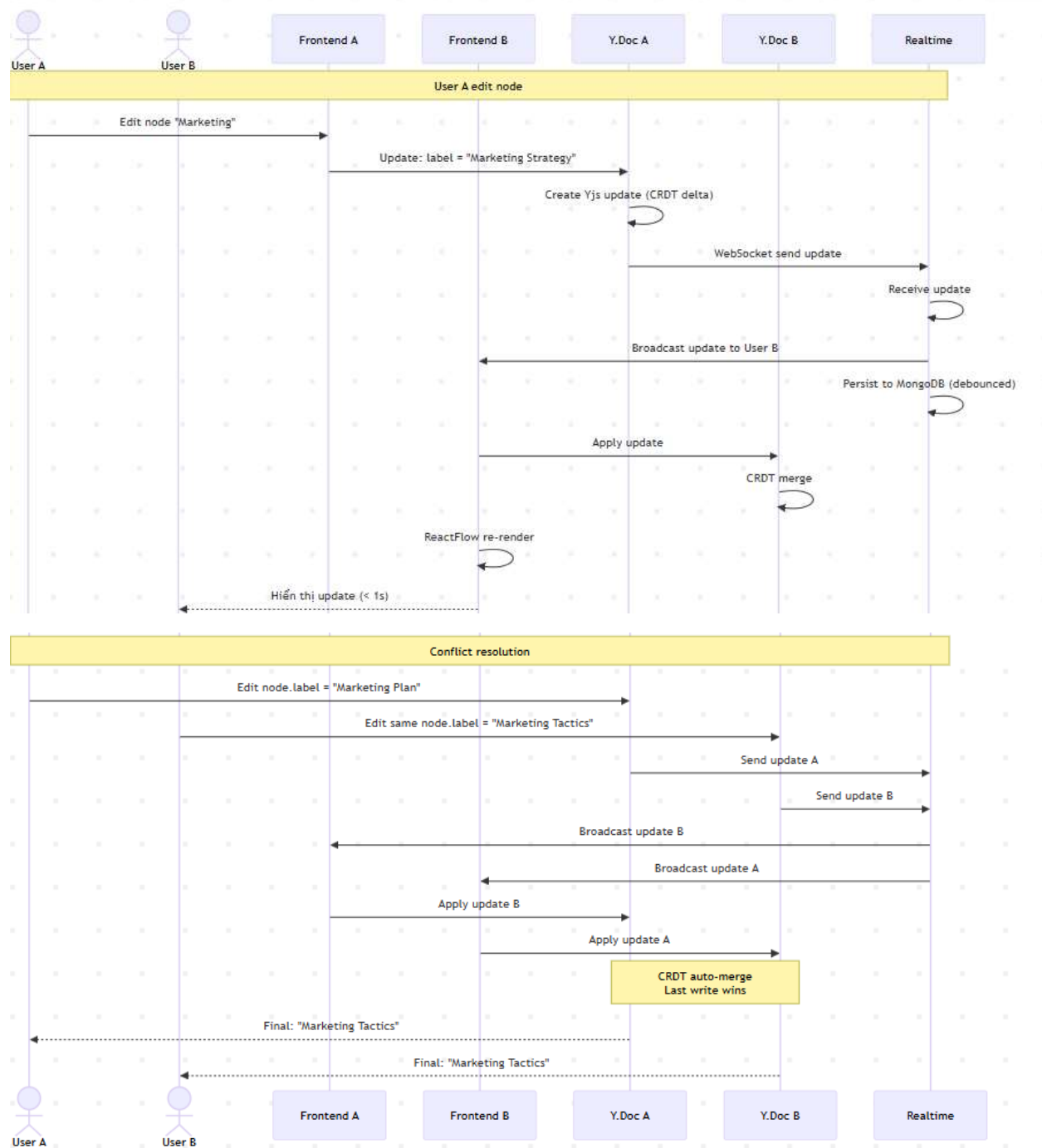
3.4.5. UC-05: Xem danh sách mindmaps



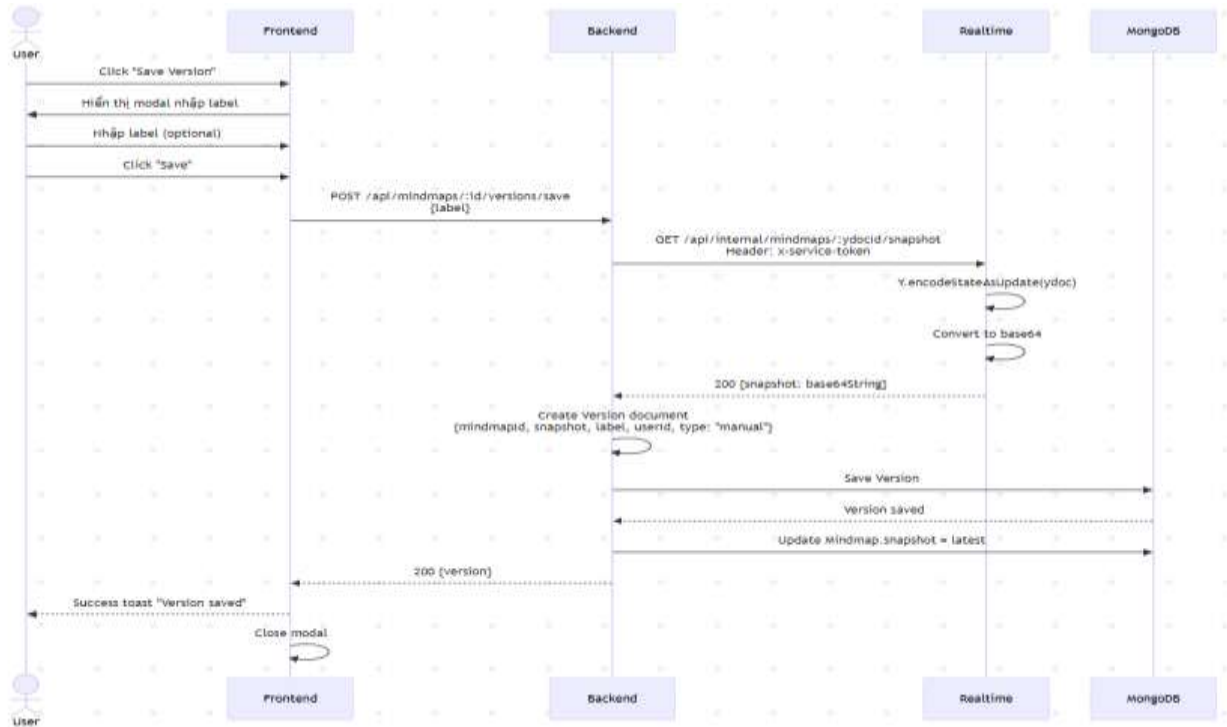
3.4.6. UC-06: Vẽ mindmap



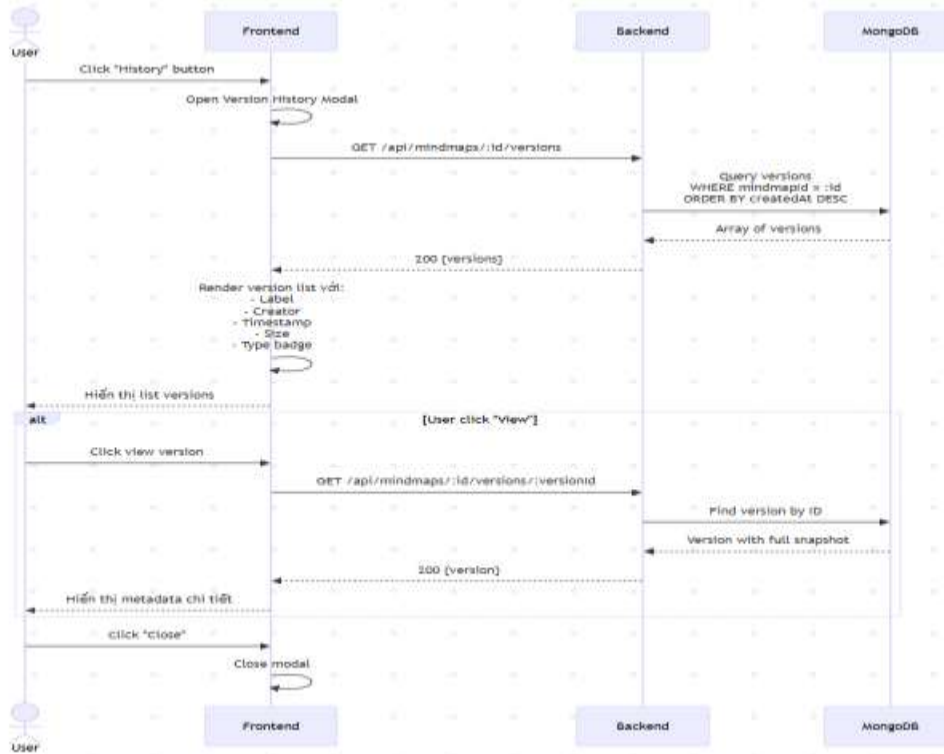
3.4.7. UC-07: Real-time collaboration



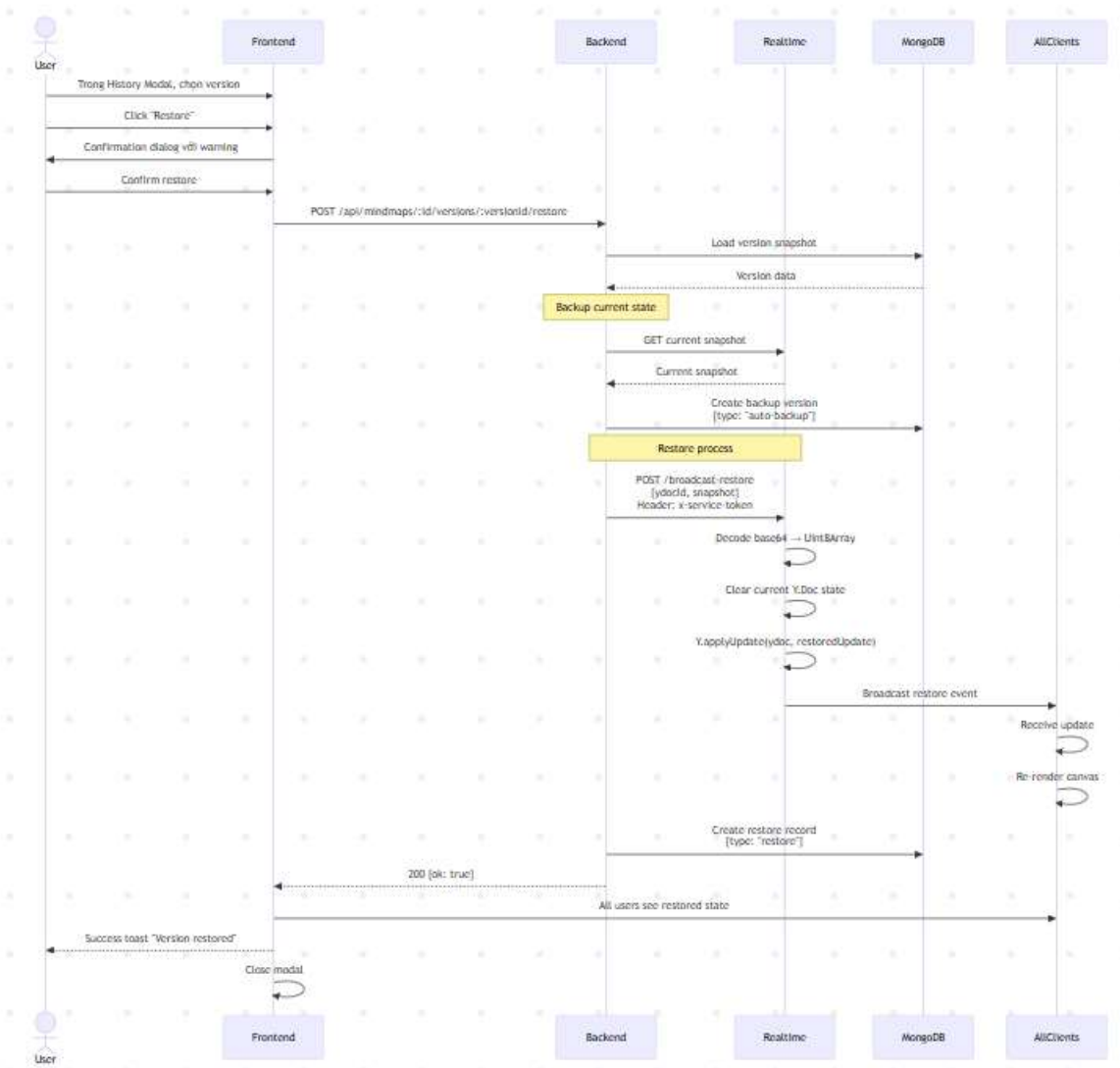
3.4.8. UC-08: Lưu version



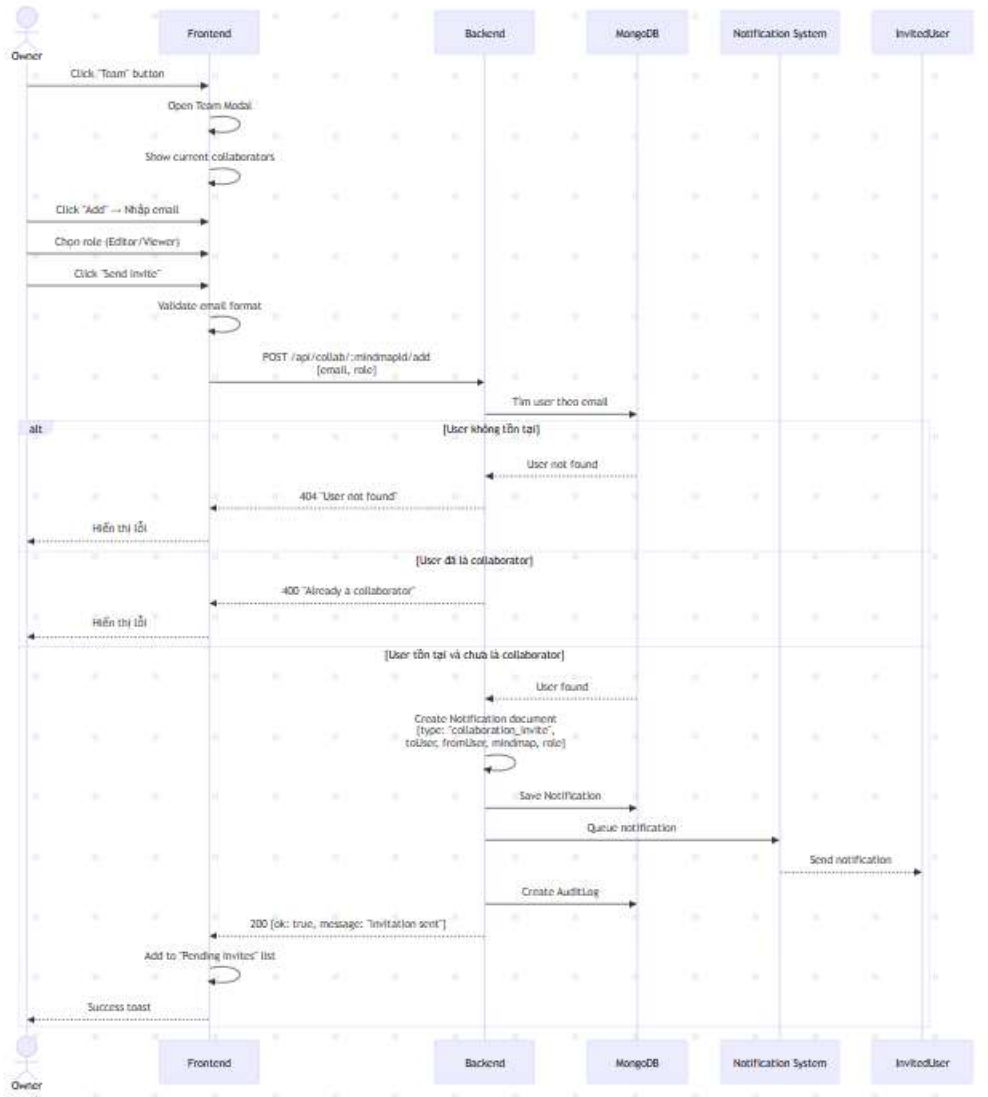
3.4.9. UC-09: Xem version history



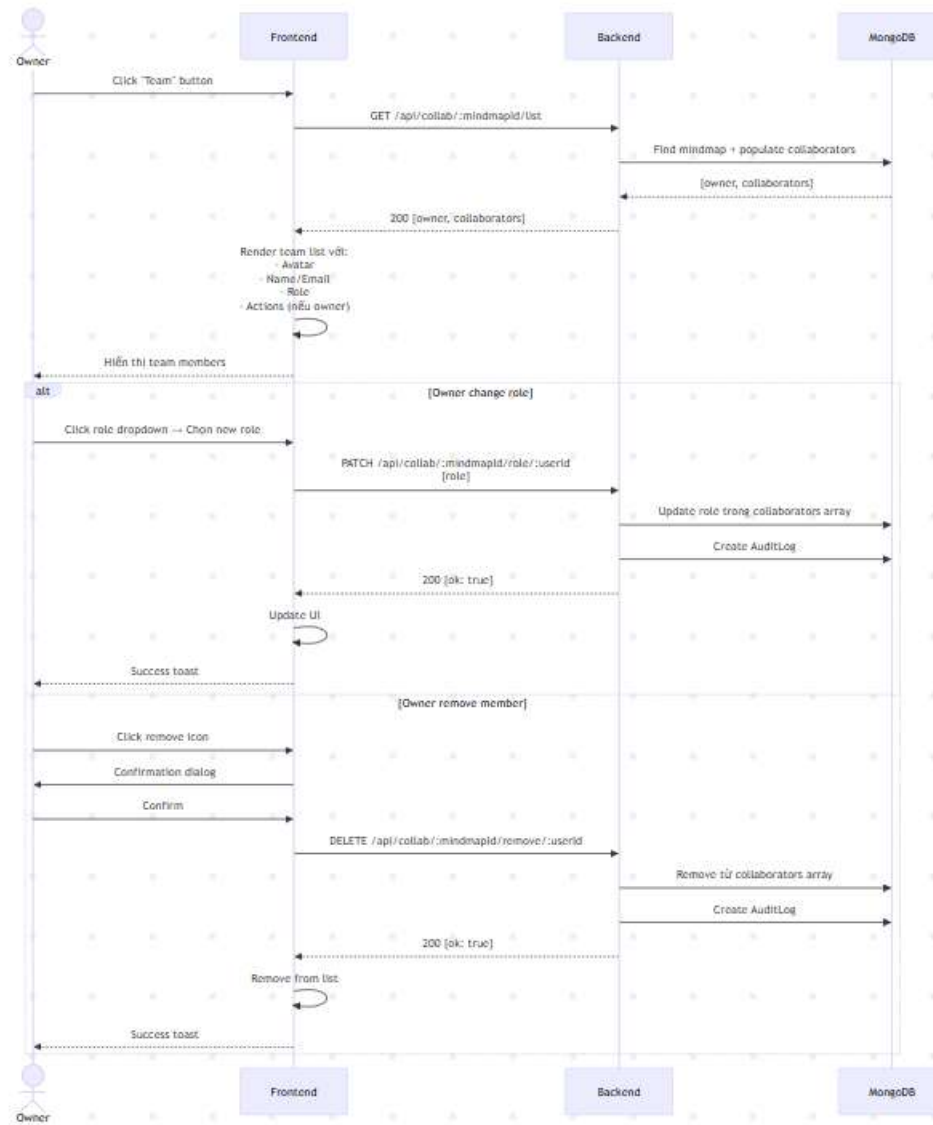
3.4.10. UC-10: Restore version



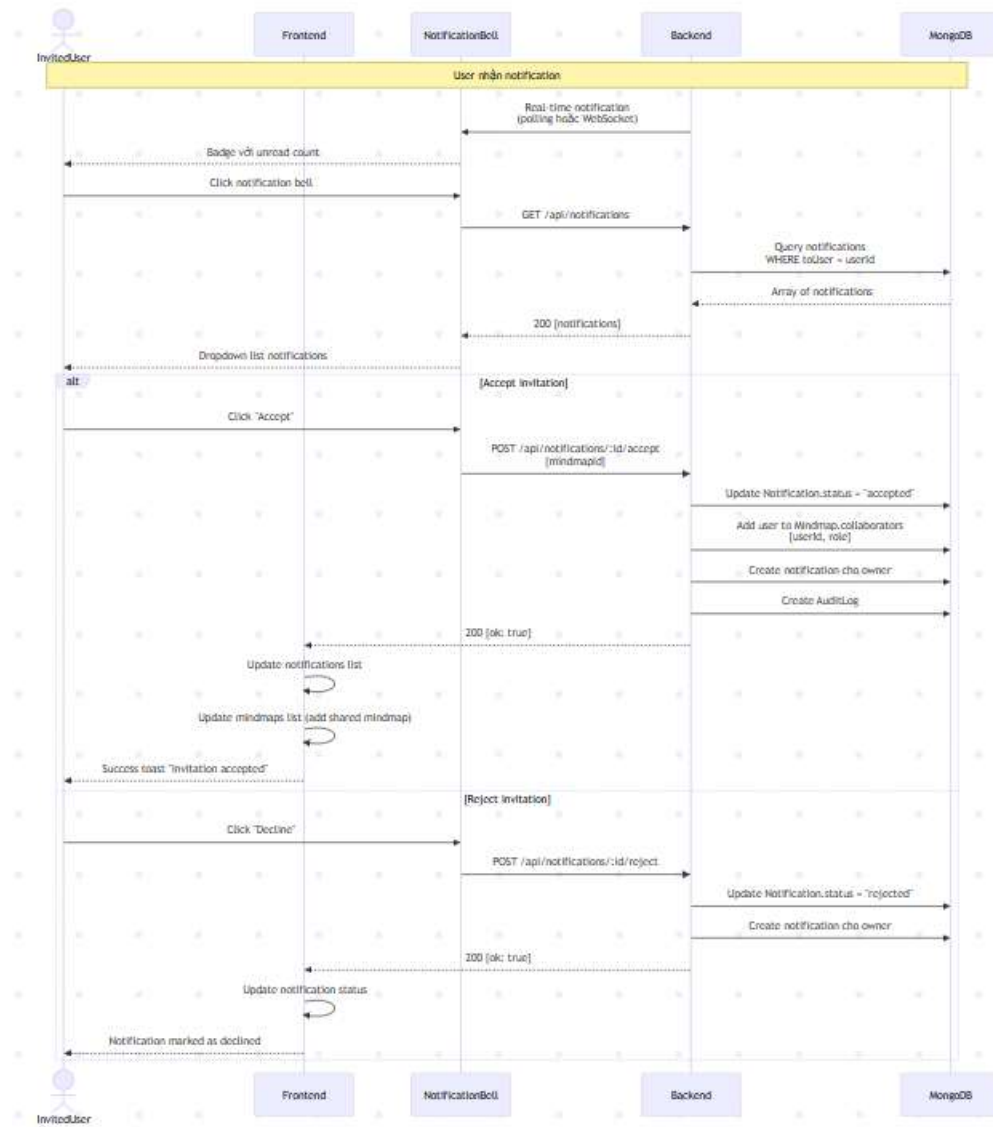
3.4.11. UC-11: Mời collaborators



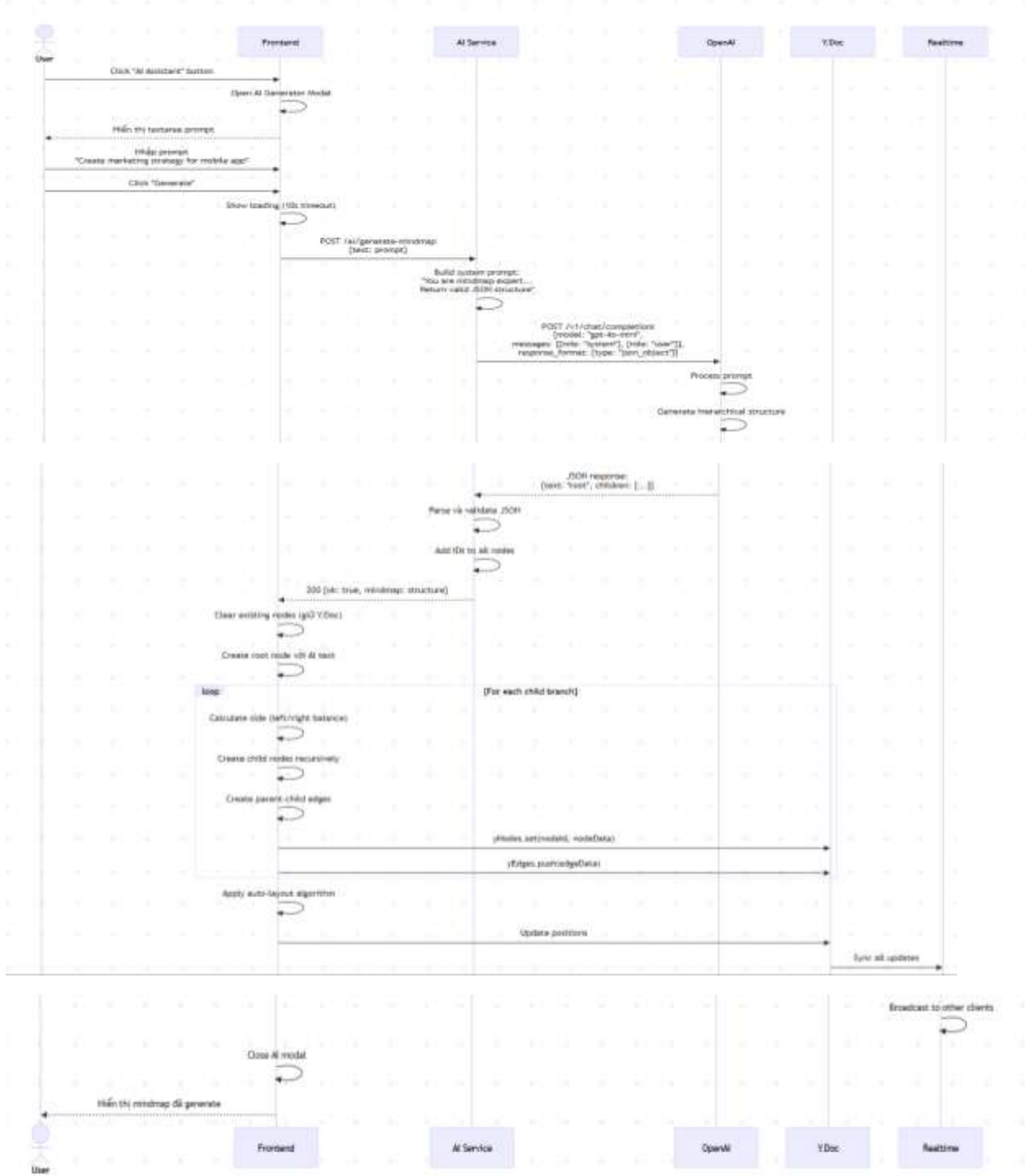
3.4.12. UC-12: Quản lý team members



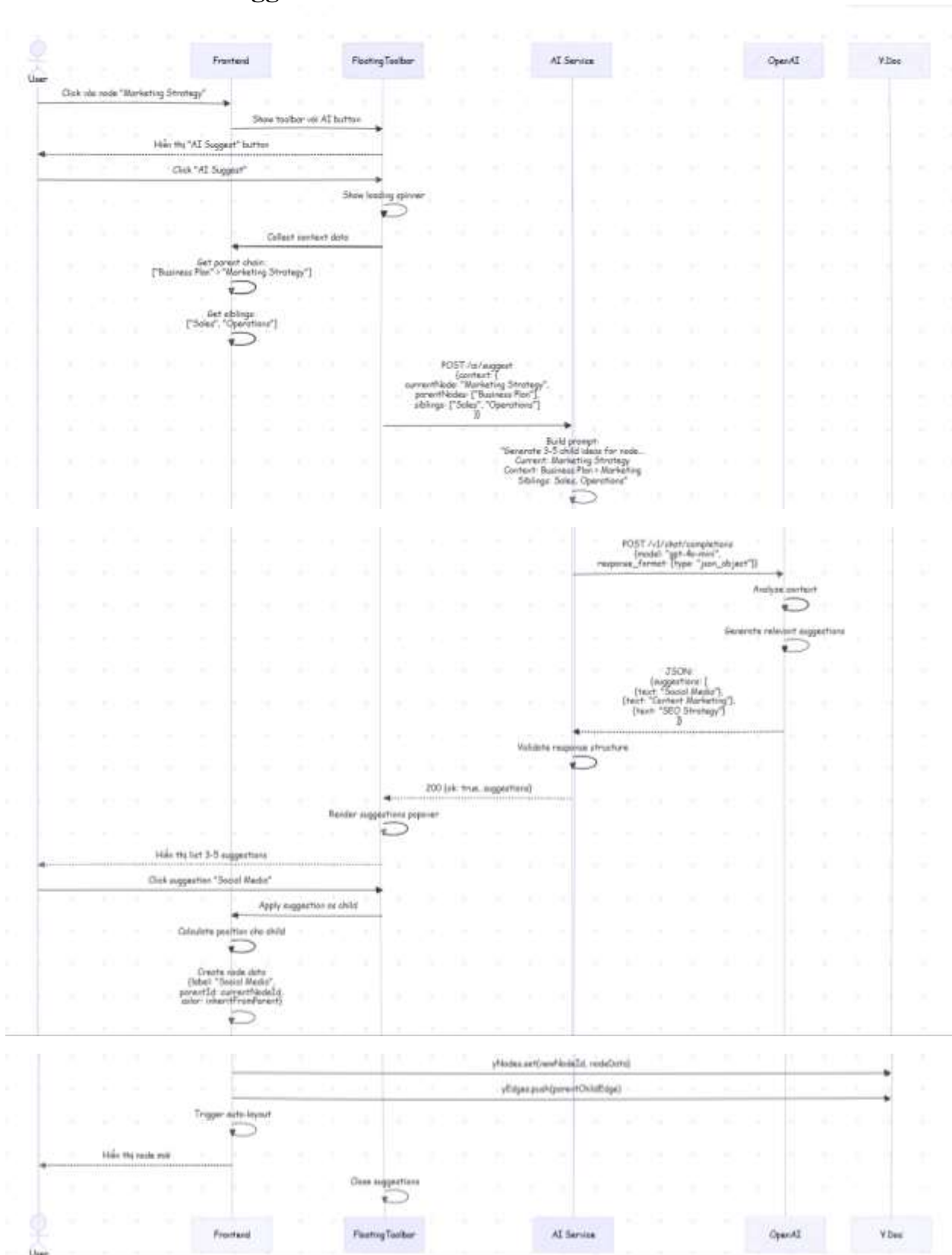
3.4.13. UC-13: Accept/Reject invite



3.4.14. UC-14: Generate mindmap với AI



3.4.15. UC-15: AI suggest nodes



3.5. Thiết kế cơ sở dữ liệu

3.5.1. Tổng quan Database

Hệ thống sử dụng MongoDB (NoSQL) với 6 collections chính để lưu trữ dữ liệu. MongoDB được chọn vì:

- Linh hoạt với schema (phù hợp cho Yjs snapshot - dữ liệu binary)
- Hiệu năng cao với large documents
- Dễ dàng scale horizontally
- Hỗ trợ tốt cho real-time applications

3.5.2. User Collection

Mục đích: Lưu trữ thông tin tài khoản người dùng

Fields chính:

- `_id`: ObjectId (Primary Key, auto-generated)
- `email`: String, unique, lowercase, indexed - Email đăng nhập
- `password`: String - Password đã hash bằng bcrypt (10 rounds)
- `name`: String - Tên hiển thị của user
- `avatarUrl`: String - URL hoặc base64 của ảnh đại diện
- `provider`: String (default: 'local') - Phương thức đăng ký
- `googleId`: String, sparse index - ID từ Google OAuth (sẽ phát triển thêm)
- `createdAt`, `updatedAt`: Date - Timestamps tự động

Indexes:

- Primary: email (unique)
- Secondary: googleId (sparse - chỉ index khi có giá trị)

Ràng buộc:

- Email phải unique và lowercase
- Password tối thiểu 6 ký tự (validation ở application layer)

3.5.3. Mindmap Collection

Mục đích: Lưu trữ metadata của mindmap và danh sách collaborators

Fields chính:

- `_id`: ObjectId (Primary Key)

- title: String, required - Tiêu đề mindmap
- description: String - Mô tả chi tiết
- ownerId: ObjectId, ref 'User', indexed - Người sở hữu
- ydocId: String, unique, indexed - ID của Yjs document (dùng cho WebSocket)
- collaborators: Array of embedded documents:
 - userId: ObjectId, ref 'User' - ID của collaborator
 - role: String (enum: 'editor', 'viewer') - Quyền hạn
- snapshot: Buffer - Binary data của Yjs state (persistence)
- createdAt, updatedAt: Date - Timestamps

Indexes:

- Primary: `_id`
- Unique: ydocId (để tìm mindmap từ WebSocket connection)
- Compound: ownerId + updatedAt (descending) - Query mindmaps của user
- Array: collaborators.userId - Query shared mindmaps

Ràng buộc:

- ydocId phải unique (generated bằng nanoid)
- ownerId không được null
- Collaborator không thể trùng với owner
- Role chỉ có 2 giá trị: 'editor' hoặc 'viewer'

3.5.4. Version Collection

Mục đích: Lưu trữ các snapshots của mindmap theo thời gian (version control)

Fields chính:

- `_id`: ObjectId (Primary Key)
- mindmapId: ObjectId, ref 'Mindmap', indexed - Mindmap tương ứng
- snapshot: Object - Cấu trúc snapshot với schema validation:
 - schemaVersion: Number - Version của snapshot format
 - encodedState: String - Yjs state encoded as base64
 - meta: Object - Metadata về snapshot (createdBy, reason, clientCount)

- createdAt: Date - Thời điểm tạo snapshot
- userId: ObjectId, ref 'User' - Người tạo version (null nếu auto-save)
- type: String (enum: 'manual', 'auto', 'restore', 'delete-backup') - Loại version
- label: String - Nhãn user tự đặt (cho manual saves)
- size: Number - Kích thước snapshot tính bằng bytes
- createdAt: Date - Thời điểm tạo version

Indexes:

- Compound: mindmapId + createdAt (descending) - List versions của mindmap

Ràng buộc:

- snapshot.encodedState không được null
- type phải thuộc enum đã định nghĩa
- size phải > 0

Quy tắc business:

- Auto-save mỗi 10 giây sau khi có thay đổi
- Manual save do user trigger
- Restore tự động tạo backup trước khi restore
- Delete-backup tạo trước khi xóa mindmap

3.5.5. Notification Collection

Mục đích: Quản lý thông báo và collaboration invites

Fields chính:

- _id: ObjectId (Primary Key)
- type: String (enum: 'collaboration_invite', 'system', 'mention') - Loại notification
- toUser: ObjectId, ref 'User', indexed - Người nhận
- fromUser: ObjectId, ref 'User' - Người gửi (null cho system)
- mindmap: ObjectId, ref 'Mindmap' - Mindmap liên quan
- role: String (enum: 'editor', 'viewer') - Role được mời (cho invite)
- message: String - Nội dung thông báo
- status: String (enum: 'pending', 'accepted', 'rejected') - Trạng thái invite

- read: Boolean, default false - Đã đọc hay chưa
- createdAt, updatedAt: Date - Timestamps

Indexes:

- Compound: toUser + read + createdAt (descending) - Unread notifications
- Compound: toUser + status - Pending invitations

Ràng buộc:

- toUser không được null
- status chỉ apply cho type 'collaboration_invite'
- Invitation không thể gửi cho chính mình

Workflow:

1. Owner mời collaborator → Tạo notification status='pending'
2. User accept → Update status='accepted', add vào Mindmap.collaborators
3. User reject → Update status='rejected'

3.5.6. RefreshToken Collection

Mục đích: Quản lý refresh tokens cho authentication

Fields chính:

- _id: ObjectId (Primary Key)
- userId: ObjectId, ref 'User', indexed - User sở hữu token
- token: String, unique, indexed - Random string (40 bytes hex)
- expiresAt: Date, indexed - Thời điểm hết hạn (7 ngày)
- createdAt: Date - Thời điểm tạo

Indexes:

- Unique: token (để verify nhanh)
- Standard: userId (list tokens của user)
- TTL Index: expiresAt với expireAfterSeconds=0 (MongoDB tự động xóa expired tokens)

Security:

- Token rotation: Mỗi lần refresh, old token bị xóa và tạo new token
- Expiry: 7 ngày, phải login lại sau đó

- One-time use: Token chỉ dùng 1 lần rồi bị replace

3.5.7. AuditLog Collection

Mục đích: Ghi lại lịch sử các actions quan trọng (auditing & debugging)

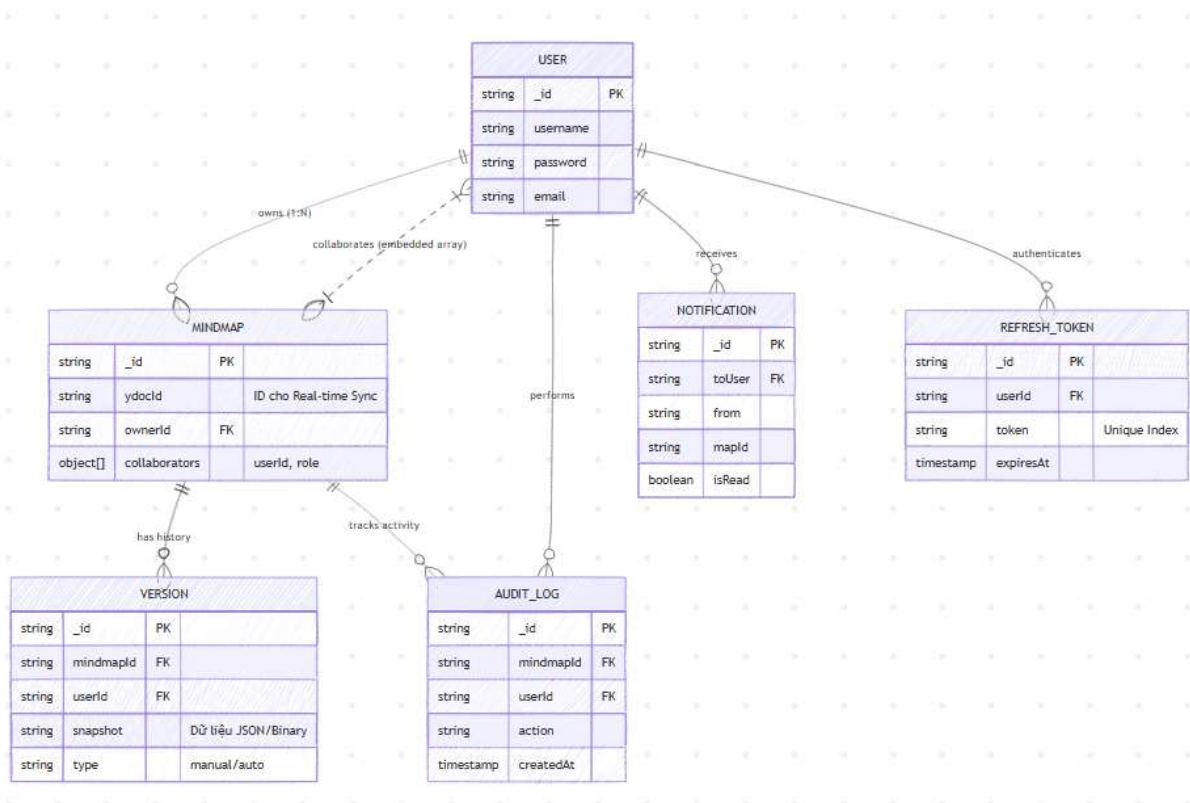
Fields chính:

- `_id`: ObjectId (Primary Key)
- `mindmapId`: ObjectId, ref 'Mindmap', indexed - Mindmap bị tác động
- `userId`: ObjectId, ref 'User', indexed - User thực hiện action
- `action`: String, enum - Loại action (create-mindmap, delete-mindmap, restore-version, etc.)
- `detail`: Mixed Object - Chi tiết cụ thể của action (flexible schema)
- `createdAt`: Date, indexed - Thời điểm xảy ra

Actions được log:

- create-mindmap, update-mindmap, delete-mindmap
- save-version, restore-version

3.5.8. Database Relationships Diagram

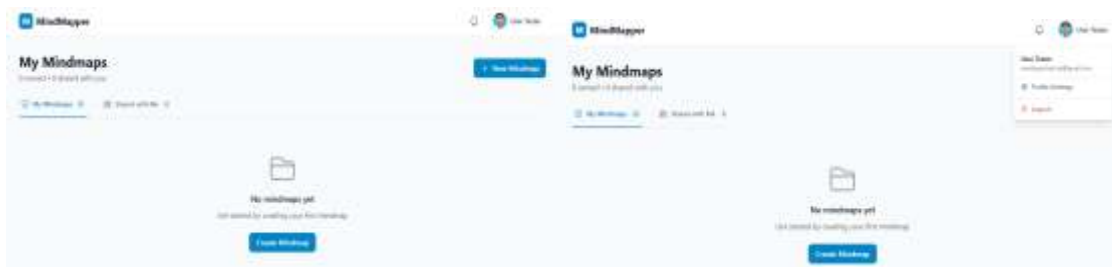


3.6. Thiết kế giao diện UI

3.6.1. Đăng nhập, đăng ký

The image displays two side-by-side web forms for the MindMapper application. The left form is for login, titled 'Welcome back' with the subtitle 'Sign in to continue to MindMapper'. It features input fields for 'Email Address' (pre-filled with 'your_email@gmail.com') and 'Password' (masked with dots). There are checkboxes for 'Remember me' and a link for 'Forgot password?'. A blue 'Sign in' button is at the bottom, with a link 'Don't have an account? Sign up for free' below it. The right form is for registration, titled 'Create your account' with the subtitle 'Start mapping your ideas today'. It includes a 'Get started' button with a plus icon. Below is a 'Get started' button with a plus icon. The form has input fields for 'Full Name *', 'Email Address *', 'Password *', and 'Confirm Password *'. A blue 'Create account' button is at the bottom, with a link 'Already have an account? Sign in' below it.

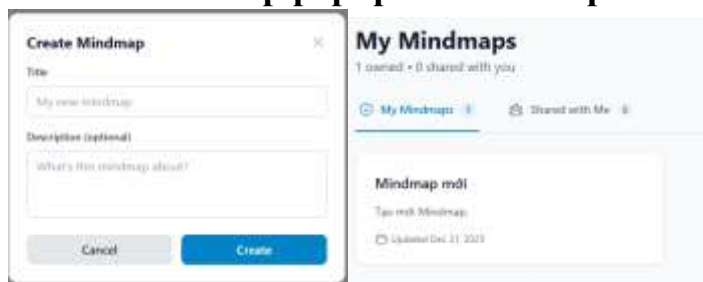
3.6.2. Dashboard



3.6.3. Profile Page

The image displays two side-by-side web forms for the MindMapper application, both titled 'Profile Settings' with the subtitle 'Manage your account settings and preferences'. The left form is for 'Profile Information' and includes a 'Profile Picture' section with a 'Choose a profile picture' button, and input fields for 'Full Name' and 'Email Address'. A blue 'Save Changes' button is at the bottom. The right form is for 'Change Password' and includes input fields for 'Current Password', 'New Password', and 'Confirm New Password'. A blue 'Update Password' button is at the bottom.

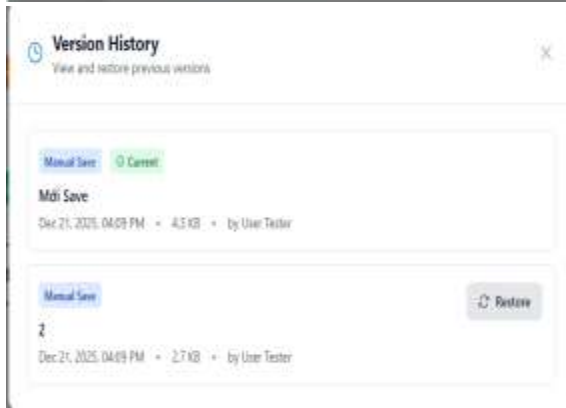
3.6.4. Create Mindmap popup và Mindmap Card ở Dashboard



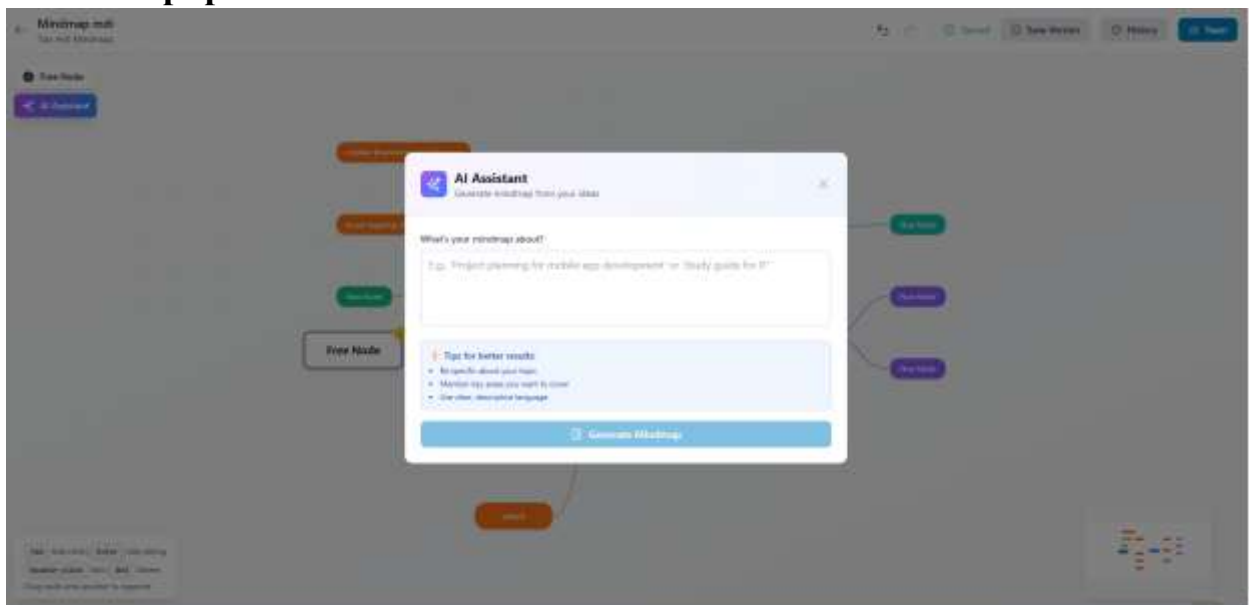
3.6.5. Editor Page khi bấm vào MindmapCard



3.6.6. Save version popup, thông báo và history



3.6.7. Popup AI Assistant



3.6.8. Thanh công cụ của Node

Lần lượt từ trái sang phải là :

Lock : Tắt tự động căn chỉnh và cho phép drag node

B : In đậm

I : In nghiêng

U: Gạch chân

A :Fontsize

[]:Chỉnh màu node

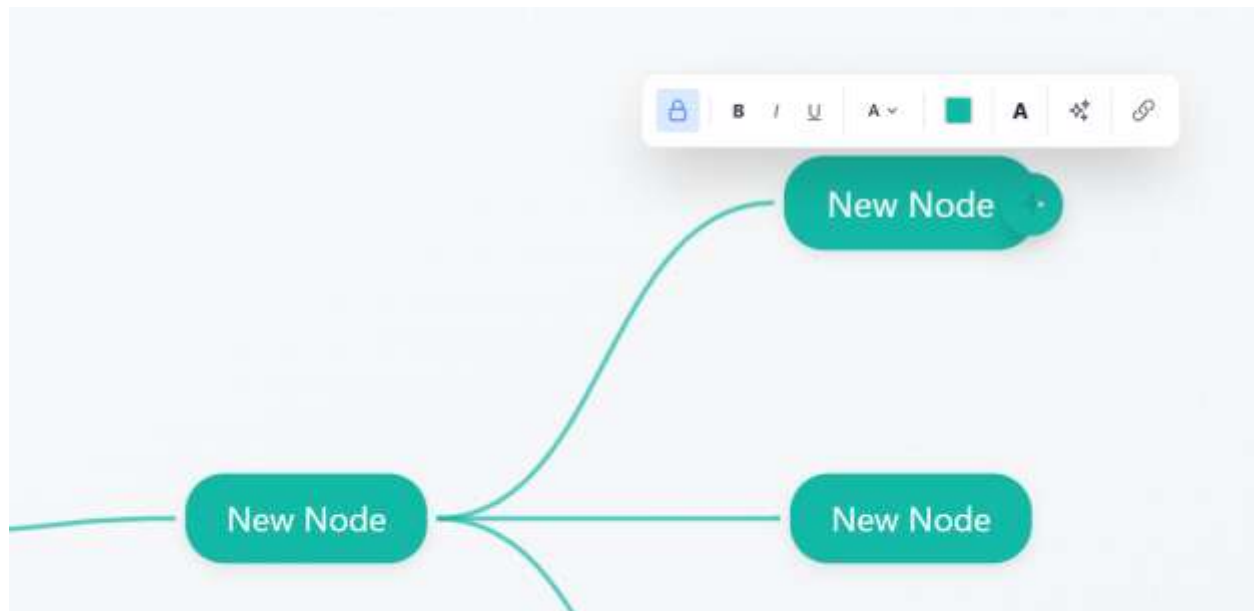
A:Màu chữ

Sparkle : AI gợi ý node

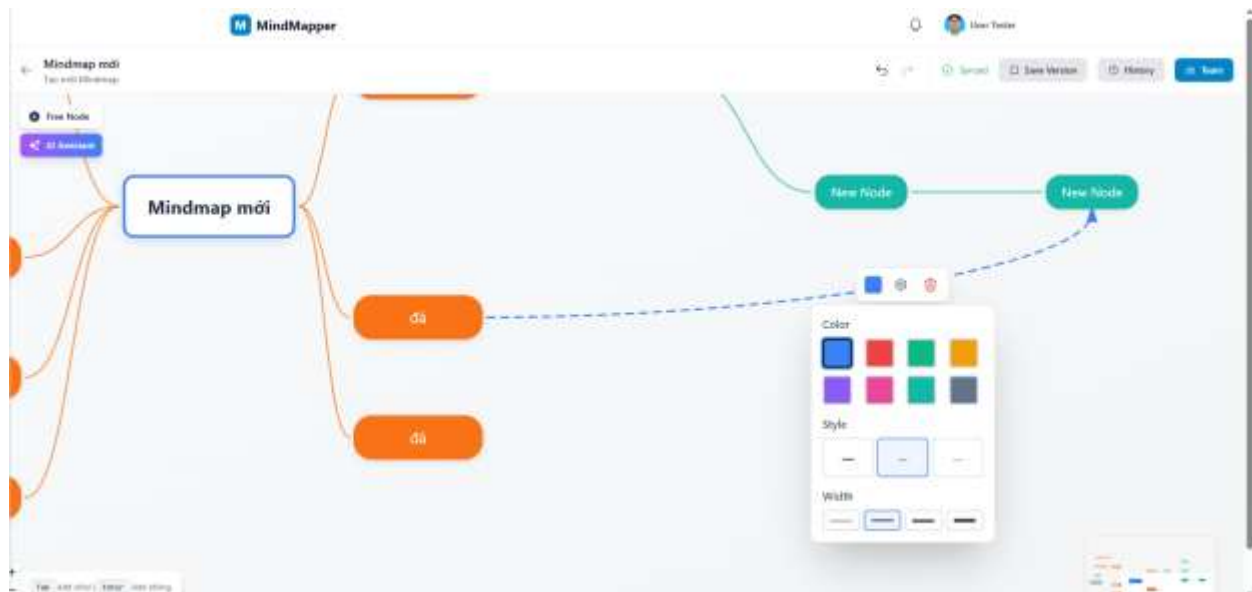
Connection : Tạo custom connection

+ :

Tạo node con



3.6.9. Chỉnh custom connection



Chương 4 : Triển khai hệ thống

4.1. Môi trường phát triển

4.1.1. Hệ điều hành

- **Development OS:** Windows/Linux/macOS (Cross-platform compatible)
- **Node.js version:** 18.x trở lên
- **MongoDB version:** 6.0+

4.1.2. Công nghệ và thư viện chính

Frontend:

- **Framework:** React 18.2 với Vite 5.0
- **UI Library:** React Flow 12.0 (mindmap visualization)
- **Styling:** Tailwind CSS 3.3
- **State Management:**
 - Zustand 4.4 (global state)
 - React Query 5.8 (server state)
- **Real-time:** Yjs 13.6 (CRDT algorithm) và y-websocket 1.5
- **Routing:** React Router DOM 6.20
- **Icons:** Heroicons 2.0, Lucide React 0.263

Backend:

- **Framework:** Express.js 4.18
- **Runtime:** Node.js (ES Modules)
- **Database ODM:** Mongoose 7.3
- **Authentication:**
 - JWT (jsonwebtoken 9.0)
 - bcryptjs 2.4 (password hashing)
- **Validation:** Joi 17.9
- **File Upload:** Multer 2.0
- **HTTP Client:** Axios 1.4

Realtime Server:

- **WebSocket Library:** ws 8.18

- **CRDT Engine:** Yjs 13.6
- **Y-WebSocket:** 2.0.4
- **Protocol:** y-protocols 1.0

AI Service:

- **AI Provider:** OpenAI GPT-4o-Mini
- **Client Library:** openai 6.15
- **Framework:** Express.js 4.18

Database:

- **Primary Database:** MongoDB 6.0+
- **Connection:** Mongoose with connection pooling

4.1.3. Công cụ hỗ trợ phát triển

IDE & Editor: Visual Studio Code

Version Control: Git, Github

Development Tools:

- **Package Manager:** npm/yarn
- **Build Tool:** Vite (Frontend)
- **Process Manager:** nodemon (development)
- **API Testing:** Postman Client
- **Database GUI:** MongoDB Compass

4.2. Giao diện người dùng (Đã có ở phần thiết kế)

Chương 5: Kết luận và hướng phát triển

5.1. Kết luận

Hiện tại, việc tổ chức ý tưởng và cộng tác trong thời gian thực đang trở nên ngày càng quan trọng, đặc biệt khi nhu cầu làm việc từ xa và làm việc nhóm ngày càng tăng. MindMapper không chỉ là một nền tảng đơn thuần giúp người dùng vẽ sơ đồ tư duy mà còn là một ứng dụng tích hợp công nghệ real-time collaboration và AI, tạo ra trải nghiệm làm việc mới mẻ và hiệu quả.

Với việc sử dụng các công nghệ tiên tiến như Yjs CRDT và WebSocket, MindMapper có khả năng đồng bộ hóa của nhiều người dùng, xử lý xung đột tự động và đảm bảo tính nhất quán của dữ liệu. Tích hợp AI OpenAI GPT-4 giúp tự động sinh sơ đồ từ văn bản và gợi ý ý tưởng thông minh, từ đó cung cấp công cụ hữu ích cho người dùng. Điều này không chỉ giúp tiết kiệm thời gian mà còn nâng cao chất lượng sơ đồ tư duy và thúc đẩy sự sáng tạo. Các tính năng như version control, save và instant restore giúp người dùng yên tâm làm việc mà không lo mất dữ liệu.

Hệ thống notification real-time và audit logging đảm bảo người dùng luôn cập nhật về các hoạt động quan trọng và có khả năng theo dõi lịch sử thay đổi. Undo/Redo với Yjs UndoManager giúp người dùng tự tin thực hiện các thao tác mà không lo sai sót. Tính năng auto-layout với thuật toán balanced tree layout đảm bảo sơ đồ luôn gọn gàng và dễ đọc.

Cuối cùng, MindMapper không chỉ dừng lại ở việc vẽ sơ đồ tư duy mà còn đóng góp vào việc số hóa quy trình làm việc, hỗ trợ quản lý tri thức và khuyến khích phong cách làm việc cộng tác hiện đại. Việc đầu tư vào nền tảng này không chỉ tạo ra lợi ích về năng suất mà còn nâng cao chất lượng làm việc nhóm, góp phần xây dựng văn hóa cộng tác và sáng tạo trong tổ chức.

5.2. Định hướng phát triển

Với sự phát triển nhanh chóng của công nghệ và nhu cầu tăng cao về công cụ cộng tác, MindMapper cần xác định lộ trình phát triển chiến lược rõ ràng.

Trong tương lai, MindMapper được định hướng phát triển nhằm đáp ứng tốt hơn nhu cầu sử dụng ngày càng đa dạng của người dùng, đặc biệt trong môi trường học tập và làm việc cộng tác. Hệ thống sẽ tiếp tục được cải tiến cả về trải nghiệm người dùng, khả năng hỗ trợ trí tuệ nhân tạo, tính năng cộng tác và hiệu năng tổng thể.

Trước hết, MindMapper tập trung nâng cao trải nghiệm người dùng thông qua việc cải tiến giao diện và mở rộng nền tảng sử dụng. Ứng dụng di động native cho iOS và Android sẽ giúp người dùng truy cập thuận tiện hơn, trong khi chế độ làm việc offline cho phép tiếp tục chỉnh sửa mindmap ngay cả khi không có kết nối mạng. Bên cạnh đó, việc bổ sung các mẫu sơ đồ (templates) cho những kịch bản phổ biến cùng chế độ trình bày chuyên nghiệp sẽ giúp người dùng tiết kiệm thời gian và tăng hiệu quả sử dụng, hỗ trợ đăng nhập bằng Google OAuth.

Bên cạnh trải nghiệm người dùng, MindMapper định hướng mở rộng việc ứng dụng trí tuệ nhân tạo nhằm hỗ trợ quá trình xây dựng và tổ chức nội dung. Các tính năng như tự động tối ưu bố cục, gợi ý nội dung thông minh theo ngữ cảnh, phân loại và tóm tắt nội dung sẽ giúp người dùng

phát triển ý tưởng nhanh chóng và trực quan hơn. Ngoài ra, hỗ trợ đa ngôn ngữ thông qua AI cũng góp phần mở rộng khả năng sử dụng của hệ thống.

Ngoài ra, MindMapper sẽ tiếp tục phát triển các tính năng cộng tác nhằm nâng cao hiệu quả làm việc nhóm. Hệ thống hướng tới việc bổ sung các công cụ trao đổi như bình luận, chú thích và gắn thẻ người dùng, cũng như mở rộng không gian làm việc nhóm để theo dõi hoạt động và phối hợp hiệu quả hơn. Các chế độ trình bày và chia sẻ thời gian thực sẽ hỗ trợ tốt cho các buổi thảo luận và thuyết trình.

Cuối cùng, việc tối ưu hiệu năng là một định hướng quan trọng trong quá trình phát triển lâu dài. MindMapper sẽ được cải tiến để xử lý các sơ đồ tư duy có quy mô lớn với số lượng node lớn mà vẫn đảm bảo khả năng hiển thị mượt mà và ổn định, từ đó nâng cao trải nghiệm tổng thể cho người dùng.

Tài liệu tham khảo

- [1] Yjs Documentation, "CRDT framework for collaborative applications," [Online]. Available: <https://docs.yjs.dev/>
- [2] React Flow, "Node-based editors and diagrams," [Online]. Available: <https://reactflow.dev/>
- [3] MongoDB Documentation, "Developer data platform," [Online]. Available: <https://www.mongodb.com/docs/>
- [4] OpenAI API, "GPT-4 Documentation," [Online]. Available: <https://platform.openai.com/docs/>
- [5] Express.js, "Web framework for Node.js," [Online]. Available: <https://expressjs.com/>
- [6] WebSocket Protocol, "RFC 6455," [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6455>
- [7] Vercel Documentation, "Frontend cloud platform," [Online]. Available: <https://vercel.com/docs>
- [8] Tailwind CSS, "Utility-first CSS framework," [Online]. Available: <https://tailwindcss.com/docs>